

“Design and Development of an Integrated Society Management System”

A project report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Department of Computer Science Engineering
&
Department of Computer Science and Information Technology

Submitted by

STUDENT NAME 1	ROLL NUMBER 1
STUDENT NAME 2	ROLL NUMBER 2
STUDENT NAME 3	ROLL NUMBER 3

Under the esteemed guidance of

GUIDE NAME



DECLARATION



We hereby declare that the project report entitled “**Design and Development of an Integrated Society Management System**” is a record of bona fide work carried out by the undersigned students in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology at K L (Deemed to be) University.

We further declare that the implementation, analysis, diagrams, testing material, and conclusions presented in this report are based on our project work. Material obtained from documentation and books has been acknowledged in the references. This report has not been submitted, in whole or in part, for the award of another degree or diploma.

Student	Registration Number
STUDENT NAME 1	ROLL NUMBER 1
STUDENT NAME 2	ROLL NUMBER 2
STUDENT NAME 3	ROLL NUMBER 3

Place: _____

Date: _____

CERTIFICATE



This is to certify that the project report entitled “**Design and Development of an Integrated Society Management System**” is a bona fide record of work completed by **STUDENT NAME 1 (ROLL NUMBER 1), STUDENT NAME 2 (ROLL NUMBER 2), STUDENT NAME 3 (ROLL NUMBER 3)** under my supervision during the academic year 2026-2027.

The work presented satisfies the academic requirements of the project and demonstrates the design, implementation, and evaluation of a full-stack society management application using React, Express, and MySQL.

Signature of the Guide

Signature of the HOD

Course Coordinator

External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, **GUIDE NAME**, for continuous guidance, technical review, and constructive suggestions throughout the design and development of Haven Society Management. The project benefited from careful discussions about database integrity, secure authentication, role separation, and usable interface design.

We thank the faculty and staff of Department of Computer Science Engineering and Department of Computer Science and Information Technology at K L (Deemed to be) University for providing the facilities and academic environment required to complete this work. We also acknowledge our classmates and reviewers whose feedback helped identify practical workflows for administrators, residents, security personnel, and maintenance teams.

Finally, we thank our families and friends for their patience and support. Their encouragement helped us complete the implementation, documentation, and verification activities represented in this report.

STUDENT NAME 1	ROLL NUMBER 1
STUDENT NAME 2	ROLL NUMBER 2
STUDENT NAME 3	ROLL NUMBER 3

ABSTRACT

Haven Society Management is a full-stack database management application that brings core residential-community operations into one secure, role-aware web system.

The platform manages flats, residents, visitors, staff, maintenance bills, payments, complaints, notices, amenities, and amenity bookings. A React single-page application provides reusable data tables, forms, status indicators, dashboards, and responsive navigation. An Express API validates JSON requests, authenticates users with JSON Web Tokens, applies role and record-ownership rules, and executes parameterized queries against MySQL.

The database schema uses primary keys, foreign keys, enumerated statuses, uniqueness constraints, and transaction boundaries to preserve consistency. Passwords are stored as bcrypt hashes; database credentials remain on the server; and residents receive scoped access to records associated with their identity or flat. Administrators receive aggregate dashboards rendered with Recharts, while operational roles receive focused views suited to their responsibilities.

The implementation demonstrates how a declarative resource model can reduce repetition across ten CRUD modules without weakening authorization. The project includes setup scripts, seed data, a backward-compatible migration, and a Postman collection covering database health, authentication, and a complete notice lifecycle. The result is a maintainable academic DBMS project with a clear path toward notifications, payment gateways, audit trails, and production deployment.

Keywords: Society management, React, Express, MySQL, JWT, RBAC, CRUD, relational database, responsive dashboard.

INDEX

No.	Chapter	Topics	Pages
1	Introduction	1.1 Background of Digital Society Administration 1.2 Problem Statement 1.3 Scope of the Project 1.4 Project Objectives 1.5 Importance of the Application 1.6 Target Users and Stakeholders 1.7 Existing Manual Process 1.8 Proposed Digital Solution 1.9 Feasibility Analysis 1.10 Benefits and Expected Outcomes 1.11 Chapter Summary	5–15
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks 2.4 Functional Requirements 2.5 Non-Functional Requirements 2.6 Requirement Traceability Summary	16–21
3	Technology Stack	3.1 Frontend: React and Vite 3.2 Frontend Routing and Component Model 3.3 CSS and Responsive Interface 3.4 Backend: Node.js and Express 3.5 REST API Design 3.6 Database: MySQL 3.7 Authentication with JWT and bcrypt 3.8 Data Visualization with Recharts 3.9 Version Control and Collaboration 3.10 External Libraries and Services 3.11 Technology Selection Summary	22–32
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Layered Architecture and Request Flow 4.3 Security and Authorization Architecture 4.4 Deployment Architecture	33–36
5	Design	5.1 Entity Relationship Diagram and Database Schema	37–37
6	Implementation	6.1 Module-Wise Implementation Overview 6.2 Flat and Resident Management 6.3 Visitor and Staff Management 6.4 Maintenance Billing and Payments 6.5 Complaint and Notice Management 6.6 Amenity and Booking Management 6.7 Frontend Rendering and State Management 6.8 API Integration 6.9 Authentication and Authorization 6.10 Validation, Transactions, and Error Handling	38–47
7	Features	7.1 Main Feature Inventory 7.2 Role-Specific Dashboards 7.3 Reusable CRUD Experience 7.4 Search, Filters, and Status Views 7.5 Resident Self-Service 7.6 Administrative Analytics 7.7 Security and Data Protection 7.8 Accessibility and Responsive Design	48–55
8	Testing	8.1 Testing Strategy 8.2 Postman API Testing 8.3 Authentication and Authorization Tests 8.4 Integration Testing 8.5 Database Constraint Testing 8.6 Frontend and Responsive Testing 8.7 Test Tools and Environments 8.8 Test Cases and Expected Results 8.9 Quality Summary and Residual Risks	56–64

INDEX (CONTINUED)

No.	Chapter	Topics	Pages
9	Deployment	9.1 Deployment Overview 9.2 Local Database Preparation 9.3 Environment Configuration 9.4 Frontend Production Build 9.5 Backend Service Deployment 9.6 Database Migration and Backup 9.7 CI/CD Pipeline Design 9.8 Security Hardening and Monitoring 9.9 Rollback and Recovery Procedure	65–73
10	Challenges and Limitations	10.1 Issues Faced During Development 10.2 Relational Data and Referential Integrity 10.3 Role Permission Complexity 10.4 Frontend State Synchronization 10.5 Authentication and Session Challenges 10.6 Solutions Applied 10.7 Current Limitations 10.8 Known Risks and Mitigations 10.9 Chapter Summary	74–82
11	Future Enhancements	11.1 Planned Features 11.2 Digital Payment Gateway Integration 11.3 Notification and Communication Services 11.4 Mobile and Progressive Web Application 11.5 Advanced Analytics and Automation 11.6 Integration and Optimization Roadmap	83–88
12	Conclusion	12.1 Project Summary 12.2 Objectives Achieved 12.3 Technical Achievements 12.4 Functional Outcomes 12.5 Security and Reliability Outcomes 12.6 User Experience Outcomes 12.7 Skills Learned During Development 12.8 Social and Operational Impact 12.9 Final Conclusion	89–97
13	References	13.1 Books and Standards 13.2 Documentation and Technical Resources	98–99
14	Appendices	14.1 Application Screenshots 14.2 Sample Code Snippets 14.3 Installation and Setup Instructions 14.4 User Manual 14.5 API Catalogue and Test Data 14.6 Review Forms and Signatures	100–105

LIST OF FIGURES AND TABLES

Figures

- Figure 4.1 High-level system architecture
- Figure 4.2 Request and response flow
- Figure 4.3 Authorization boundary
- Figure 4.4 Deployment topology
- Figure 5.1 Entity relationship model
- Figure 6.1 API integration sequence
- Figure 6.2 JWT authentication sequence
- Figure 8.1 Integration test flow
- Figure 11.1 Enhancement roadmap

Tables

- Table 2.1 Hardware requirements
- Table 2.2 Software requirements
- Table 2.3 Functional requirements
- Table 3.1 Technology stack
- Table 5.1 Entity catalogue
- Table 7.1 Role capability matrix
- Table 8.1 API test cases
- Table 9.1 Environment variables
- Table A.1 Endpoint catalogue

Abbreviations

API	Application Programming Interface	JWT	JSON Web Token
RBAC	Role-Based Access Control	CRUD	Create, Read, Update, Delete
DBMS	Database Management System	SPA	Single-Page Application
UI/UX	User Interface / User Experience	CI/CD	Continuous Integration / Delivery

CHAPTER 1

1.1 BACKGROUND OF DIGITAL SOCIETY ADMINISTRATION

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on background of digital society administration and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of centralize society records. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, background of digital society administration depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support improve visibility and accountability without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Background of Digital Society Administration is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Background of Digital Society Administration is addressed through a defined application or database boundary.
- Improve visibility and accountability: Background of Digital Society Administration is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Background of Digital Society Administration is addressed through a defined application or database boundary.
- Preserve resident privacy: Background of Digital Society Administration is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Background of Digital Society Administration is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.2 PROBLEM STATEMENT

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on problem statement and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of reduce duplicate manual entry. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, problem statement depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support support distinct operational roles without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Problem Statement is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Problem Statement is addressed through a defined application or database boundary.
- Improve visibility and accountability: Problem Statement is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Problem Statement is addressed through a defined application or database boundary.
- Preserve resident privacy: Problem Statement is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Problem Statement is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.3 SCOPE OF THE PROJECT

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on scope of the project and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of improve visibility and accountability. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, scope of the project depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support preserve resident privacy without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Scope of the Project is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Scope of the Project is addressed through a defined application or database boundary.
- Improve visibility and accountability: Scope of the Project is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Scope of the Project is addressed through a defined application or database boundary.
- Preserve resident privacy: Scope of the Project is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Scope of the Project is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.4 PROJECT OBJECTIVES

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on project objectives and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of support distinct operational roles. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, project objectives depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support centralize society records without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Project Objectives is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Project Objectives is addressed through a defined application or database boundary.
- Improve visibility and accountability: Project Objectives is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Project Objectives is addressed through a defined application or database boundary.
- Preserve resident privacy: Project Objectives is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Project Objectives is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.5 IMPORTANCE OF THE APPLICATION

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on importance of the application and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of preserve resident privacy. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, importance of the application depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support reduce duplicate manual entry without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Importance of the Application is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Importance of the Application is addressed through a defined application or database boundary.
- Improve visibility and accountability: Importance of the Application is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Importance of the Application is addressed through a defined application or database boundary.
- Preserve resident privacy: Importance of the Application is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Importance of the Application is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.6 TARGET USERS AND STAKEHOLDERS

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on target users and stakeholders and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of centralize society records. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, target users and stakeholders depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support improve visibility and accountability without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Target Users and Stakeholders is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Target Users and Stakeholders is addressed through a defined application or database boundary.
- Improve visibility and accountability: Target Users and Stakeholders is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Target Users and Stakeholders is addressed through a defined application or database boundary.
- Preserve resident privacy: Target Users and Stakeholders is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Target Users and Stakeholders is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.7 EXISTING MANUAL PROCESS

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on existing manual process and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of reduce duplicate manual entry. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, existing manual process depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support distinct operational roles without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Existing Manual Process is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Existing Manual Process is addressed through a defined application or database boundary.
- Improve visibility and accountability: Existing Manual Process is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Existing Manual Process is addressed through a defined application or database boundary.
- Preserve resident privacy: Existing Manual Process is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Existing Manual Process is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.8 PROPOSED DIGITAL SOLUTION

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on proposed digital solution and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of improve visibility and accountability. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, proposed digital solution depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support preserve resident privacy without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Proposed Digital Solution is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Proposed Digital Solution is addressed through a defined application or database boundary.
- Improve visibility and accountability: Proposed Digital Solution is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Proposed Digital Solution is addressed through a defined application or database boundary.
- Preserve resident privacy: Proposed Digital Solution is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Proposed Digital Solution is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.9 FEASIBILITY ANALYSIS

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on feasibility analysis and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of support distinct operational roles. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, feasibility analysis depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support centralize society records without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Feasibility Analysis is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Feasibility Analysis is addressed through a defined application or database boundary.
- Improve visibility and accountability: Feasibility Analysis is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Feasibility Analysis is addressed through a defined application or database boundary.
- Preserve resident privacy: Feasibility Analysis is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Feasibility Analysis is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.10 BENEFITS AND EXPECTED OUTCOMES

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on benefits and expected outcomes and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of preserve resident privacy. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, benefits and expected outcomes depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support reduce duplicate manual entry without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Benefits and Expected Outcomes is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Benefits and Expected Outcomes is addressed through a defined application or database boundary.
- Improve visibility and accountability: Benefits and Expected Outcomes is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Benefits and Expected Outcomes is addressed through a defined application or database boundary.
- Preserve resident privacy: Benefits and Expected Outcomes is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Benefits and Expected Outcomes is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 1

1.11 CHAPTER SUMMARY

The application replaces fragmented registers and spreadsheets with a unified operational view of flats, residents, visitors, staff, maintenance, payments, complaints, notices, amenities, and bookings. The project is designed for residential communities where administrators require reliable records and residents need limited self-service access. This section focuses on chapter summary and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of centralize society records. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, chapter summary depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support improve visibility and accountability without introducing unnecessary architectural layers.

Design focus

- Centralize society records: Chapter Summary is addressed through an observable user or administrator workflow.
- Reduce duplicate manual entry: Chapter Summary is addressed through a defined application or database boundary.
- Improve visibility and accountability: Chapter Summary is addressed through an observable user or administrator workflow.

Acceptance evidence

- Support distinct operational roles: Chapter Summary is addressed through a defined application or database boundary.
- Preserve resident privacy: Chapter Summary is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Chapter Summary is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 2

2.1 HARDWARE REQUIREMENTS

Requirements were derived from the implemented routes, forms, database constraints, and role rules. They distinguish mandatory business behavior from quality attributes such as security, responsiveness, maintainability, consistency, and recoverability. This section focuses on hardware requirements and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of authenticated access. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, hardware requirements depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support responsive browser interface without introducing unnecessary architectural layers.

Component	Minimum	Recommended
Processor	Dual-core 2 GHz	Modern 4-core CPU
Memory	8 GB RAM	16 GB RAM
Storage	2 GB free	SSD with 10 GB free
Network	Localhost	Stable broadband for deployment
Client	1366 × 768 display	Full HD display

Page outcome: Hardware Requirements is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 2

2.2 SOFTWARE REQUIREMENTS

Requirements were derived from the implemented routes, forms, database constraints, and role rules. They distinguish mandatory business behavior from quality attributes such as security, responsiveness, maintainability, consistency, and recoverability. This section focuses on software requirements and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of persistent relational storage. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, software requirements depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support clear validation feedback without introducing unnecessary architectural layers.

Software	Role
Node.js 20+	Frontend tooling and API runtime
MySQL 8	Relational persistence and constraints
Modern browser	React client execution
VS Code	Development and debugging
Postman	Repeatable API checks
Git	Version history and collaboration

Page outcome: Software Requirements is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 2

2.3 DEVELOPMENT TOOLS AND FRAMEWORKS

Requirements were derived from the implemented routes, forms, database constraints, and role rules. They distinguish mandatory business behavior from quality attributes such as security, responsiveness, maintainability, consistency, and recoverability. This section focuses on development tools and frameworks and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of responsive browser interface. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, development tools and frameworks depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support recoverable deployment configuration without introducing unnecessary architectural layers.

Design focus

- **Authenticated access:** Development Tools and Frameworks is addressed through an observable user or administrator workflow.
- **Persistent relational storage:** Development Tools and Frameworks is addressed through a defined application or database boundary.
- **Responsive browser interface:** Development Tools and Frameworks is addressed through an observable user or administrator workflow.

Acceptance evidence

- **Clear validation feedback:** Development Tools and Frameworks is addressed through a defined application or database boundary.
- **Recoverable deployment configuration:** Development Tools and Frameworks is addressed through an observable user or administrator workflow.
- **Repository anchor:** README.md and project configuration.

Page outcome: Development Tools and Frameworks is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 2

2.4 FUNCTIONAL REQUIREMENTS

Requirements were derived from the implemented routes, forms, database constraints, and role rules. They distinguish mandatory business behavior from quality attributes such as security, responsiveness, maintainability, consistency, and recoverability. This section focuses on functional requirements and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of clear validation feedback. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, functional requirements depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support authenticated access without introducing unnecessary architectural layers.

Design focus

- Authenticated access: Functional Requirements is addressed through an observable user or administrator workflow.
- Persistent relational storage: Functional Requirements is addressed through a defined application or database boundary.
- Responsive browser interface: Functional Requirements is addressed through an observable user or administrator workflow.

Acceptance evidence

- Clear validation feedback: Functional Requirements is addressed through a defined application or database boundary.
- Recoverable deployment configuration: Functional Requirements is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Functional Requirements is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 2

2.5 NON-FUNCTIONAL REQUIREMENTS

Requirements were derived from the implemented routes, forms, database constraints, and role rules. They distinguish mandatory business behavior from quality attributes such as security, responsiveness, maintainability, consistency, and recoverability. This section focuses on non-functional requirements and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of recoverable deployment configuration. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, non-functional requirements depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support persistent relational storage without introducing unnecessary architectural layers.

Design focus

- Authenticated access: Non-Functional Requirements is addressed through an observable user or administrator workflow.
- Persistent relational storage: Non-Functional Requirements is addressed through a defined application or database boundary.
- Responsive browser interface: Non-Functional Requirements is addressed through an observable user or administrator workflow.

Acceptance evidence

- Clear validation feedback: Non-Functional Requirements is addressed through a defined application or database boundary.
- Recoverable deployment configuration: Non-Functional Requirements is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Non-Functional Requirements is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 2

2.6 REQUIREMENT TRACEABILITY SUMMARY

Requirements were derived from the implemented routes, forms, database constraints, and role rules. They distinguish mandatory business behavior from quality attributes such as security, responsiveness, maintainability, consistency, and recoverability. This section focuses on requirement traceability summary and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of authenticated access. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, requirement traceability summary depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support responsive browser interface without introducing unnecessary architectural layers.

Design focus

- **Authenticated access:** Requirement Traceability Summary is addressed through an observable user or administrator workflow.
- **Persistent relational storage:** Requirement Traceability Summary is addressed through a defined application or database boundary.
- **Responsive browser interface:** Requirement Traceability Summary is addressed through an observable user or administrator workflow.

Acceptance evidence

- **Clear validation feedback:** Requirement Traceability Summary is addressed through a defined application or database boundary.
- **Recoverable deployment configuration:** Requirement Traceability Summary is addressed through an observable user or administrator workflow.
- **Repository anchor:** README.md and project configuration.

Page outcome: Requirement Traceability Summary is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.1 FRONTEND: REACT AND VITE

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on frontend: react and vite and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of component reuse. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, frontend: react and vite depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support parameterized sql without introducing unnecessary architectural layers.

Design focus

- Component reuse: Frontend: React and Vite is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: Frontend: React and Vite is addressed through a defined application or database boundary.
- Parameterized SQL: Frontend: React and Vite is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: Frontend: React and Vite is addressed through a defined application or database boundary.
- Reproducible package versions: Frontend: React and Vite is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Frontend: React and Vite is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.2 FRONTEND ROUTING AND COMPONENT MODEL

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on frontend routing and component model and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of non-blocking http handling. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, frontend routing and component model depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support declarative resource metadata without introducing unnecessary architectural layers.

Design focus

- Component reuse: Frontend Routing and Component Model is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: Frontend Routing and Component Model is addressed through a defined application or database boundary.
- Parameterized SQL: Frontend Routing and Component Model is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: Frontend Routing and Component Model is addressed through a defined application or database boundary.
- Reproducible package versions: Frontend Routing and Component Model is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Frontend Routing and Component Model is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.3 CSS AND RESPONSIVE INTERFACE

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on css and responsive interface and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of parameterized sql. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, css and responsive interface depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support reproducible package versions without introducing unnecessary architectural layers.

Design focus

- Component reuse: CSS and Responsive Interface is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: CSS and Responsive Interface is addressed through a defined application or database boundary.
- Parameterized SQL: CSS and Responsive Interface is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: CSS and Responsive Interface is addressed through a defined application or database boundary.
- Reproducible package versions: CSS and Responsive Interface is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: CSS and Responsive Interface is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.4 BACKEND: NODE.JS AND EXPRESS

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on backend: node.js and express and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of declarative resource metadata. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, backend: node.js and express depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support component reuse without introducing unnecessary architectural layers.

Design focus

- Component reuse: Backend: Node.js and Express is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: Backend: Node.js and Express is addressed through a defined application or database boundary.
- Parameterized SQL: Backend: Node.js and Express is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: Backend: Node.js and Express is addressed through a defined application or database boundary.
- Reproducible package versions: Backend: Node.js and Express is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Backend: Node.js and Express is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.5 REST API DESIGN

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on rest api design and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of reproducible package versions. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, rest api design depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support non-blocking http handling without introducing unnecessary architectural layers.

Design focus

- Component reuse: REST API Design is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: REST API Design is addressed through a defined application or database boundary.
- Parameterized SQL: REST API Design is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: REST API Design is addressed through a defined application or database boundary.
- Reproducible package versions: REST API Design is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: REST API Design is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.6 DATABASE: MYSQL

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on database: mysql and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of component reuse. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, database: mysql depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support parameterized sql without introducing unnecessary architectural layers.

Design focus

- Component reuse: Database: MySQL is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: Database: MySQL is addressed through a defined application or database boundary.
- Parameterized SQL: Database: MySQL is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: Database: MySQL is addressed through a defined application or database boundary.
- Reproducible package versions: Database: MySQL is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Database: MySQL is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.7 AUTHENTICATION WITH JWT AND BCrypt

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on authentication with jwt and bcrypt and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of non-blocking http handling. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, authentication with jwt and bcrypt depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support declarative resource metadata without introducing unnecessary architectural layers.

Design focus

- Component reuse: Authentication with JWT and bcrypt is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: Authentication with JWT and bcrypt is addressed through a defined application or database boundary.
- Parameterized SQL: Authentication with JWT and bcrypt is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: Authentication with JWT and bcrypt is addressed through a defined application or database boundary.
- Reproducible package versions: Authentication with JWT and bcrypt is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Authentication with JWT and bcrypt is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.8 DATA VISUALIZATION WITH RECHARTS

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on data visualization with recharts and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of parameterized sql. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, data visualization with recharts depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support reproducible package versions without introducing unnecessary architectural layers.

Design focus

- Component reuse: Data Visualization with Recharts is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: Data Visualization with Recharts is addressed through a defined application or database boundary.
- Parameterized SQL: Data Visualization with Recharts is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: Data Visualization with Recharts is addressed through a defined application or database boundary.
- Reproducible package versions: Data Visualization with Recharts is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Data Visualization with Recharts is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.9 VERSION CONTROL AND COLLABORATION

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on version control and collaboration and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of declarative resource metadata. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, version control and collaboration depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support component reuse without introducing unnecessary architectural layers.

Design focus

- Component reuse: Version Control and Collaboration is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: Version Control and Collaboration is addressed through a defined application or database boundary.
- Parameterized SQL: Version Control and Collaboration is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: Version Control and Collaboration is addressed through a defined application or database boundary.
- Reproducible package versions: Version Control and Collaboration is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Version Control and Collaboration is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.10 EXTERNAL LIBRARIES AND SERVICES

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on external libraries and services and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of reproducible package versions. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, external libraries and services depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support non-blocking http handling without introducing unnecessary architectural layers.

Design focus

- Component reuse: External Libraries and Services is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: External Libraries and Services is addressed through a defined application or database boundary.
- Parameterized SQL: External Libraries and Services is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: External Libraries and Services is addressed through a defined application or database boundary.
- Reproducible package versions: External Libraries and Services is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: External Libraries and Services is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 3

3.11 TECHNOLOGY SELECTION SUMMARY

The implementation uses React 19.2.8, React Router 7.18.3, Vite 8.2.2, Express 5.2.1, MySQL through mysql2, JSON Web Tokens, bcryptjs, Lucide icons, and Recharts. The selected stack keeps the browser, API, and relational database responsibilities explicit. This section focuses on technology selection summary and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of component reuse. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, technology selection summary depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support parameterized sql without introducing unnecessary architectural layers.

Design focus

- Component reuse: Technology Selection Summary is addressed through an observable user or administrator workflow.
- Non-blocking HTTP handling: Technology Selection Summary is addressed through a defined application or database boundary.
- Parameterized SQL: Technology Selection Summary is addressed through an observable user or administrator workflow.

Acceptance evidence

- Declarative resource metadata: Technology Selection Summary is addressed through a defined application or database boundary.
- Reproducible package versions: Technology Selection Summary is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Technology Selection Summary is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

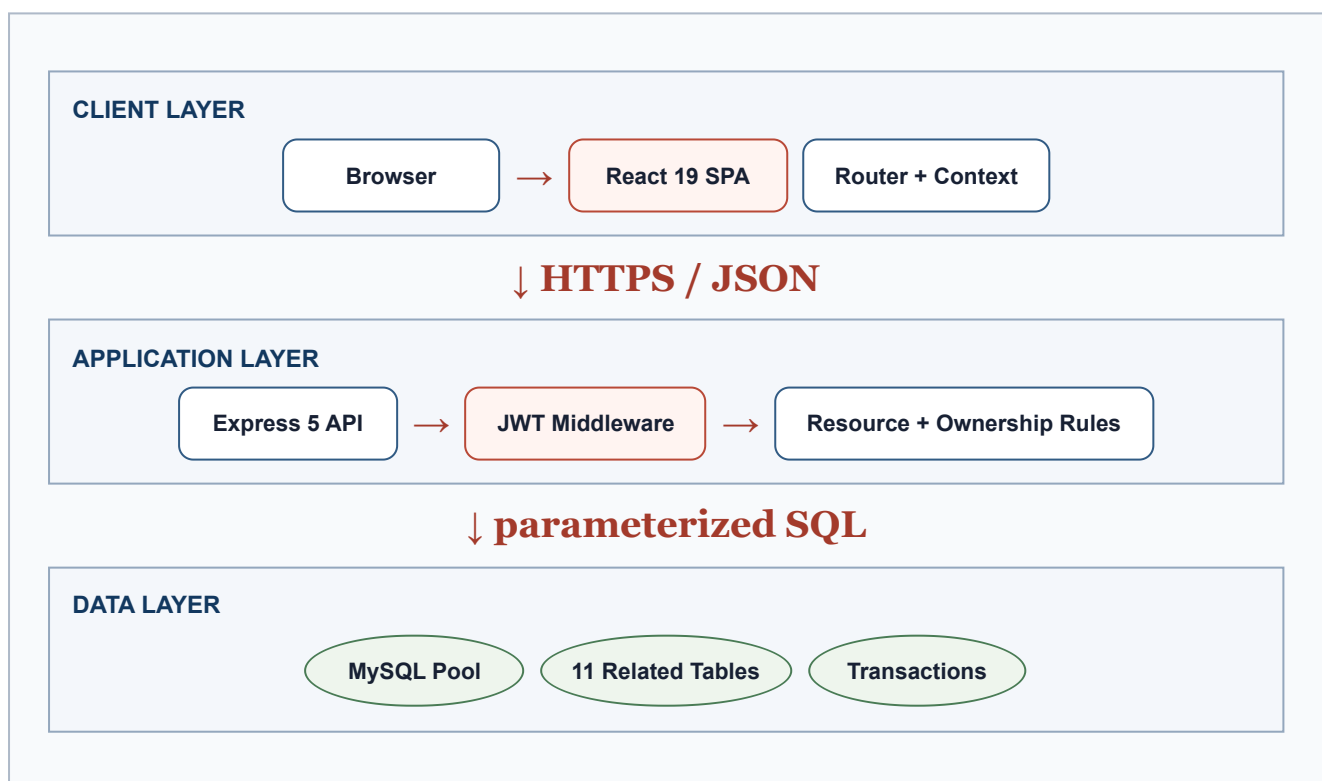
CHAPTER 4

4.1 HIGH-LEVEL ARCHITECTURE DIAGRAM

Haven follows a three-tier web architecture. The React single-page application calls an Express REST API; the API authenticates requests, applies role and ownership rules, and executes parameterized statements against MySQL. The browser never receives database credentials. This section focuses on high-level architecture diagram and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of presentation layer. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, high-level architecture diagram depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support authorization boundary without introducing unnecessary architectural layers.



Page outcome: High-Level Architecture Diagram is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

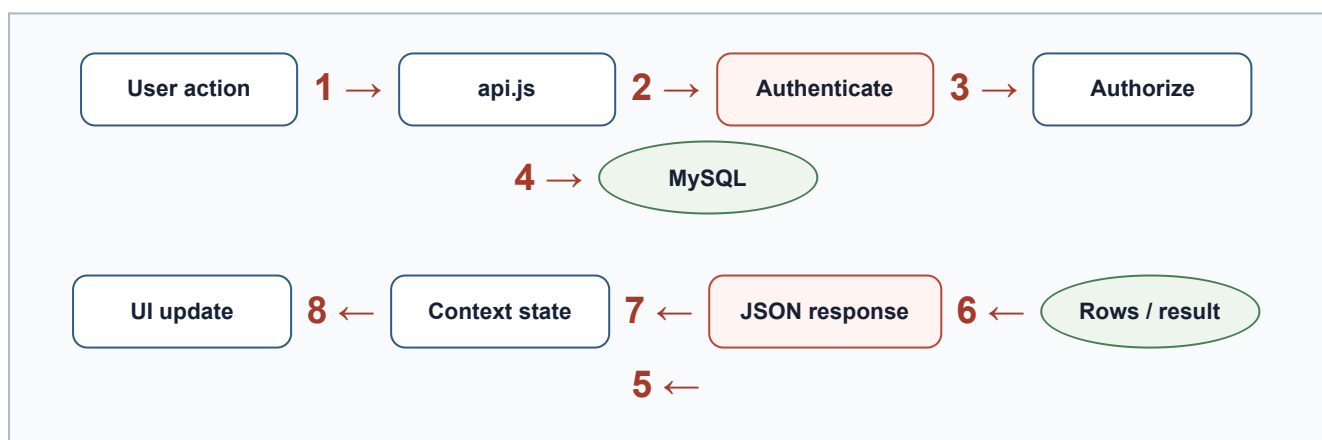
CHAPTER 4

4.2 LAYERED ARCHITECTURE AND REQUEST FLOW

Haven follows a three-tier web architecture. The React single-page application calls an Express REST API; the API authenticates requests, applies role and ownership rules, and executes parameterized statements against MySQL. The browser never receives database credentials. This section focuses on layered architecture and request flow and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of application service layer. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, layered architecture and request flow depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support relational persistence layer without introducing unnecessary architectural layers.



Page outcome: Layered Architecture and Request Flow is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

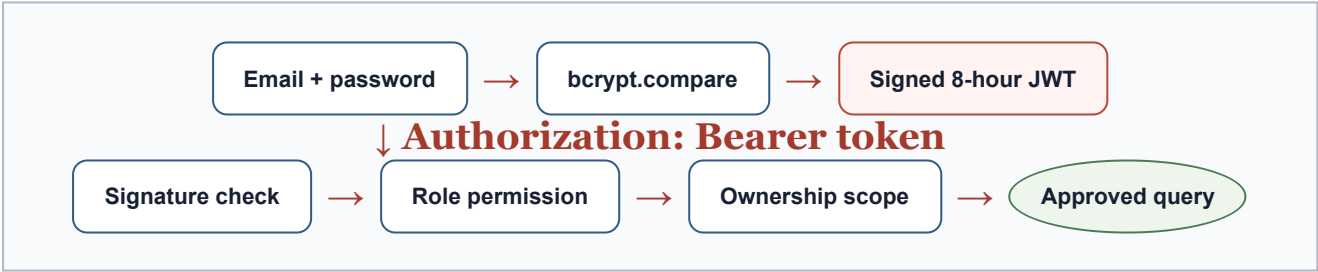
CHAPTER 4

4.3 SECURITY AND AUTHORIZATION ARCHITECTURE

Haven follows a three-tier web architecture. The React single-page application calls an Express REST API; the API authenticates requests, applies role and ownership rules, and executes parameterized statements against MySQL. The browser never receives database credentials. This section focuses on security and authorization architecture and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of authorization boundary. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, security and authorization architecture depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support operational health endpoint without introducing unnecessary architectural layers.



Page outcome: Security and Authorization Architecture is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

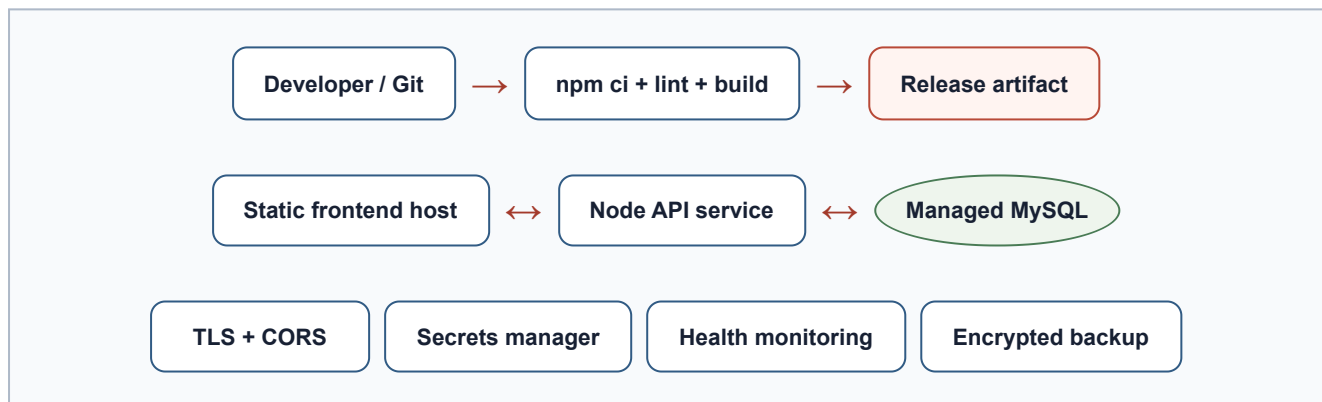
CHAPTER 4

4.4 DEPLOYMENT ARCHITECTURE

Haven follows a three-tier web architecture. The React single-page application calls an Express REST API; the API authenticates requests, applies role and ownership rules, and executes parameterized statements against MySQL. The browser never receives database credentials. This section focuses on deployment architecture and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of relational persistence layer. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, deployment architecture depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support presentation layer without introducing unnecessary architectural layers.



Page outcome: Deployment Architecture is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 5

5.1 ENTITY RELATIONSHIP DIAGRAM AND DATABASE SCHEMA

The relational schema contains flats, residents, staff, visitors, maintenance bills, payments, complaints, notices, amenities, amenity bookings, and application users. Foreign keys preserve identity and ownership while unique keys prevent duplicate business records. This section focuses on entity relationship diagram and database schema and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of primary and foreign keys. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, entity relationship diagram and database schema depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support enumerated status values without introducing unnecessary architectural layers.

<table><tr><th>FLATS</th></tr><tr><td>PK flat_id</td></tr><tr><td>UQ block + number</td></tr><tr><td>type, occupancy</td></tr></table>	FLATS	PK flat_id	UQ block + number	type, occupancy	<table><tr><th>RESIDENTS</th></tr><tr><td>PK resident_id</td></tr><tr><td>FK flat_id</td></tr><tr><td>email, role, type</td></tr></table>	RESIDENTS	PK resident_id	FK flat_id	email, role, type	<table><tr><th>APP_USERS</th></tr><tr><td>PK user_id</td></tr><tr><td>FK resident/staff/flat</td></tr><tr><td>hash, role, active</td></tr></table>	APP_USERS	PK user_id	FK resident/staff/flat	hash, role, active
FLATS														
PK flat_id														
UQ block + number														
type, occupancy														
RESIDENTS														
PK resident_id														
FK flat_id														
email, role, type														
APP_USERS														
PK user_id														
FK resident/staff/flat														
hash, role, active														
<table><tr><th>VISITORS</th></tr><tr><td>PK visitor_id</td></tr><tr><td>FK flat_id</td></tr><tr><td>entry, exit, status</td></tr></table>	VISITORS	PK visitor_id	FK flat_id	entry, exit, status	<table><tr><th>STAFF</th></tr><tr><td>PK staff_id</td></tr><tr><td>name, phone</td></tr><tr><td>staff_type</td></tr></table>	STAFF	PK staff_id	name, phone	staff_type	<table><tr><th>MAINTENANCE_BILLS</th></tr><tr><td>PK bill_id</td></tr><tr><td>FK flat_id</td></tr><tr><td>month, amount, status</td></tr></table>	MAINTENANCE_BILLS	PK bill_id	FK flat_id	month, amount, status
VISITORS														
PK visitor_id														
FK flat_id														
entry, exit, status														
STAFF														
PK staff_id														
name, phone														
staff_type														
MAINTENANCE_BILLS														
PK bill_id														
FK flat_id														
month, amount, status														
<table><tr><th>PAYMENTS</th></tr><tr><td>PK payment_id</td></tr><tr><td>FK bill + resident</td></tr><tr><td>method, transaction</td></tr></table>	PAYMENTS	PK payment_id	FK bill + resident	method, transaction	<table><tr><th>COMPLAINTS</th></tr><tr><td>PK complaint_id</td></tr><tr><td>FK resident_id</td></tr><tr><td>title, status</td></tr></table>	COMPLAINTS	PK complaint_id	FK resident_id	title, status	<table><tr><th>NOTICES</th></tr><tr><td>PK notice_id</td></tr><tr><td>title, description</td></tr><tr><td>notice_date</td></tr></table>	NOTICES	PK notice_id	title, description	notice_date
PAYMENTS														
PK payment_id														
FK bill + resident														
method, transaction														
COMPLAINTS														
PK complaint_id														
FK resident_id														
title, status														
NOTICES														
PK notice_id														
title, description														
notice_date														
<table><tr><th>AMENITIES</th></tr><tr><td>PK amenity_id</td></tr><tr><td>UQ amenity_name</td></tr><tr><td>location, status</td></tr></table>	AMENITIES	PK amenity_id	UQ amenity_name	location, status	<table><tr><th>AMENITY_BOOKINGS</th></tr><tr><td>PK booking_id</td></tr><tr><td>FK resident + amenity</td></tr><tr><td>date, time, status</td></tr></table>	AMENITY_BOOKINGS	PK booking_id	FK resident + amenity	date, time, status					
AMENITIES														
PK amenity_id														
UQ amenity_name														
location, status														
AMENITY_BOOKINGS														
PK booking_id														
FK resident + amenity														
date, time, status														

Relationships: Flat 1—N Residents; Flat 1—N Visitors; Flat 1—N Bills; Resident 1—N Payments; Resident 1—N Complaints; Resident N—M Amenities through Amenity Bookings; App User optionally maps one-to-one to a resident or staff record.

CHAPTER 6

6.1 MODULE-WISE IMPLEMENTATION OVERVIEW

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on module-wise implementation overview and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of list and summary views. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, module-wise implementation overview depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support transactional writes without introducing unnecessary architectural layers.

Design focus • List and summary views: Module-Wise Implementation Overview is addressed through an observable user or administrator workflow. • Validated create/edit forms: Module-Wise Implementation Overview is addressed through a defined application or database boundary. • Transactional writes: Module-Wise Implementation Overview is addressed through an observable user or administrator workflow.	Acceptance evidence • Server-side record scoping: Module-Wise Implementation Overview is addressed through a defined application or database boundary. • Consistent errors and toasts: Module-Wise Implementation Overview is addressed through an observable user or administrator workflow. • Repository anchor: frontend/src and backend/index.js.
--	---

Page outcome: Module-Wise Implementation Overview is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 6

6.2 FLAT AND RESIDENT MANAGEMENT

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on flat and resident management and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of validated create/edit forms. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, flat and resident management depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support server-side record scoping without introducing unnecessary architectural layers.

Design focus

- List and summary views: Flat and Resident Management is addressed through an observable user or administrator workflow.
- Validated create/edit forms: Flat and Resident Management is addressed through a defined application or database boundary.
- Transactional writes: Flat and Resident Management is addressed through an observable user or administrator workflow.

Acceptance evidence

- Server-side record scoping: Flat and Resident Management is addressed through a defined application or database boundary.
- Consistent errors and toasts: Flat and Resident Management is addressed through an observable user or administrator workflow.
- Repository anchor: frontend/src and backend/index.js.

Page outcome: Flat and Resident Management is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 6

6.3 VISITOR AND STAFF MANAGEMENT

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on visitor and staff management and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of transactional writes. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, visitor and staff management depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support consistent errors and toasts without introducing unnecessary architectural layers.

Design focus

- List and summary views: Visitor and Staff Management is addressed through an observable user or administrator workflow.
- Validated create/edit forms: Visitor and Staff Management is addressed through a defined application or database boundary.
- Transactional writes: Visitor and Staff Management is addressed through an observable user or administrator workflow.

Acceptance evidence

- Server-side record scoping: Visitor and Staff Management is addressed through a defined application or database boundary.
- Consistent errors and toasts: Visitor and Staff Management is addressed through an observable user or administrator workflow.
- Repository anchor: frontend/src and backend/index.js.

Page outcome: Visitor and Staff Management is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 6

6.4 MAINTENANCE BILLING AND PAYMENTS

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on maintenance billing and payments and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of server-side record scoping. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, maintenance billing and payments depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support list and summary views without introducing unnecessary architectural layers.

Design focus

- List and summary views: Maintenance Billing and Payments is addressed through an observable user or administrator workflow.
- Validated create/edit forms: Maintenance Billing and Payments is addressed through a defined application or database boundary.
- Transactional writes: Maintenance Billing and Payments is addressed through an observable user or administrator workflow.

Acceptance evidence

- Server-side record scoping: Maintenance Billing and Payments is addressed through a defined application or database boundary.
- Consistent errors and toasts: Maintenance Billing and Payments is addressed through an observable user or administrator workflow.
- Repository anchor: frontend/src and backend/index.js.

Page outcome: Maintenance Billing and Payments is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 6

6.5 COMPLAINT AND NOTICE MANAGEMENT

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on complaint and notice management and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of consistent errors and toasts. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, complaint and notice management depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support validated create/edit forms without introducing unnecessary architectural layers.

Design focus

- List and summary views: Complaint and Notice Management is addressed through an observable user or administrator workflow.
- Validated create/edit forms: Complaint and Notice Management is addressed through a defined application or database boundary.
- Transactional writes: Complaint and Notice Management is addressed through an observable user or administrator workflow.

Acceptance evidence

- Server-side record scoping: Complaint and Notice Management is addressed through a defined application or database boundary.
- Consistent errors and toasts: Complaint and Notice Management is addressed through an observable user or administrator workflow.
- Repository anchor: frontend/src and backend/index.js.

Page outcome: Complaint and Notice Management is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 6

6.6 AMENITY AND BOOKING MANAGEMENT

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on amenity and booking management and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of list and summary views. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, amenity and booking management depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support transactional writes without introducing unnecessary architectural layers.

Design focus

- List and summary views: Amenity and Booking Management is addressed through an observable user or administrator workflow.
- Validated create/edit forms: Amenity and Booking Management is addressed through a defined application or database boundary.
- Transactional writes: Amenity and Booking Management is addressed through an observable user or administrator workflow.

Acceptance evidence

- Server-side record scoping: Amenity and Booking Management is addressed through a defined application or database boundary.
- Consistent errors and toasts: Amenity and Booking Management is addressed through an observable user or administrator workflow.
- Repository anchor: frontend/src and backend/index.js.

Page outcome: Amenity and Booking Management is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 6

6.7 FRONTEND RENDERING AND STATE MANAGEMENT

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on frontend rendering and state management and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of validated create/edit forms. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, frontend rendering and state management depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support server-side record scoping without introducing unnecessary architectural layers.

Design focus

- List and summary views: Frontend Rendering and State Management is addressed through an observable user or administrator workflow.
- Validated create/edit forms: Frontend Rendering and State Management is addressed through a defined application or database boundary.
- Transactional writes: Frontend Rendering and State Management is addressed through an observable user or administrator workflow.

Acceptance evidence

- Server-side record scoping: Frontend Rendering and State Management is addressed through a defined application or database boundary.
- Consistent errors and toasts: Frontend Rendering and State Management is addressed through an observable user or administrator workflow.
- Repository anchor: frontend/src and backend/index.js.

Page outcome: Frontend Rendering and State Management is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

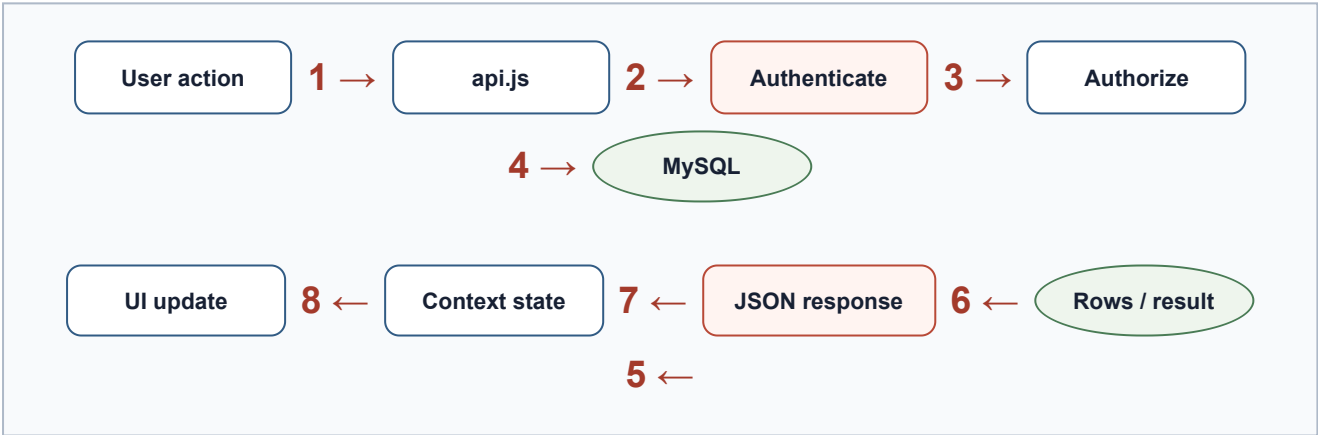
CHAPTER 6

6.8 API INTEGRATION

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on api integration and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of transactional writes. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, api integration depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support consistent errors and toasts without introducing unnecessary architectural layers.



Page outcome: API Integration is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

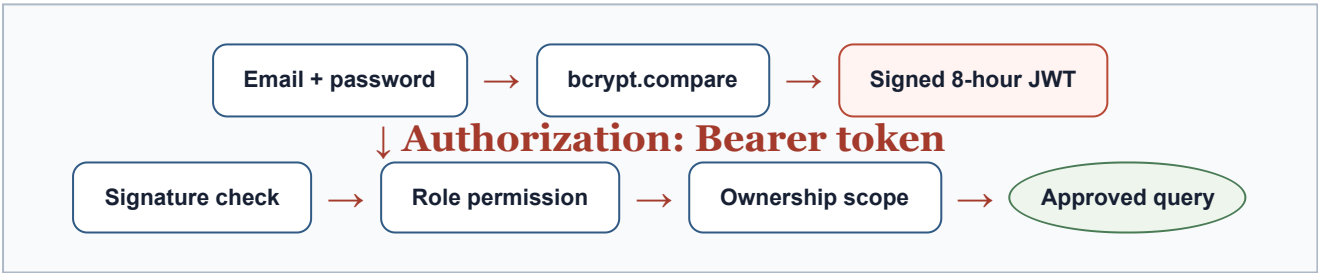
CHAPTER 6

6.9 AUTHENTICATION AND AUTHORIZATION

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on authentication and authorization and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of server-side record scoping. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, authentication and authorization depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support list and summary views without introducing unnecessary architectural layers.



Page outcome: Authentication and Authorization is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 6

6.10 VALIDATION, TRANSACTIONS, AND ERROR HANDLING

Implementation is organized around shared CRUD abstractions. Frontend resource metadata defines fields, columns, filters, and status summaries. Backend resource metadata maps approved fields to tables. This reduces duplication while keeping authorization authoritative on the server. This section focuses on validation, transactions, and error handling and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of consistent errors and toasts. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, validation, transactions, and error handling depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support validated create/edit forms without introducing unnecessary architectural layers.

Design focus

- List and summary views: Validation, Transactions, and Error Handling is addressed through an observable user or administrator workflow.
- Validated create/edit forms: Validation, Transactions, and Error Handling is addressed through a defined application or database boundary.
- Transactional writes: Validation, Transactions, and Error Handling is addressed through an observable user or administrator workflow.

Acceptance evidence

- Server-side record scoping: Validation, Transactions, and Error Handling is addressed through a defined application or database boundary.
- Consistent errors and toasts: Validation, Transactions, and Error Handling is addressed through an observable user or administrator workflow.
- Repository anchor: frontend/src and backend/index.js.

Page outcome: Validation, Transactions, and Error Handling is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 7

7.1 MAIN FEATURE INVENTORY

The finished feature set supports seven application roles: Admin, Committee, Secretary, Resident, Security, Housekeeping, and Maintenance Staff. Each role sees only relevant pages and actions, while record ownership further limits resident and staff data. This section focuses on main feature inventory and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of dashboard analytics. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, main feature inventory depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support status badges without introducing unnecessary architectural layers.

Role	Primary capabilities
Admin	All modules, all actions, analytics
Committee	Operations plus residents, visitors, complaints, notices, amenities, bookings
Secretary	Resident services, visitors, complaints, notices, amenities, bookings
Resident	Own visitors, bills, payments, complaints, and bookings; notices and amenities
Security	Visitor management and notices
Housekeeping	Focused dashboard and notices
Maintenance Staff	Complaint resolution and notices

Page outcome: Main Feature Inventory is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 7

7.2 ROLE-SPECIFIC DASHBOARDS

The finished feature set supports seven application roles: Admin, Committee, Secretary, Resident, Security, Housekeeping, and Maintenance Staff. Each role sees only relevant pages and actions, while record ownership further limits resident and staff data. This section focuses on role-specific dashboards and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of resource filters. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, role-specific dashboards depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support role-specific navigation without introducing unnecessary architectural layers.

Design focus

- Dashboard analytics: Role-Specific Dashboards is addressed through an observable user or administrator workflow.
- Resource filters: Role-Specific Dashboards is addressed through a defined application or database boundary.
- Status badges: Role-Specific Dashboards is addressed through an observable user or administrator workflow.

Acceptance evidence

- Role-specific navigation: Role-Specific Dashboards is addressed through a defined application or database boundary.
- Resident self-service: Role-Specific Dashboards is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Role-Specific Dashboards is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 7

7.3 REUSABLE CRUD EXPERIENCE

The finished feature set supports seven application roles: Admin, Committee, Secretary, Resident, Security, Housekeeping, and Maintenance Staff. Each role sees only relevant pages and actions, while record ownership further limits resident and staff data. This section focuses on reusable crud experience and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of status badges. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, reusable crud experience depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support resident self-service without introducing unnecessary architectural layers.

Role	Primary capabilities
Admin	All modules, all actions, analytics
Committee	Operations plus residents, visitors, complaints, notices, amenities, bookings
Secretary	Resident services, visitors, complaints, notices, amenities, bookings
Resident	Own visitors, bills, payments, complaints, and bookings; notices and amenities
Security	Visitor management and notices
Housekeeping	Focused dashboard and notices
Maintenance Staff	Complaint resolution and notices

Page outcome: Reusable CRUD Experience is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 7

7.4 SEARCH, FILTERS, AND STATUS VIEWS

The finished feature set supports seven application roles: Admin, Committee, Secretary, Resident, Security, Housekeeping, and Maintenance Staff. Each role sees only relevant pages and actions, while record ownership further limits resident and staff data. This section focuses on search, filters, and status views and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of role-specific navigation. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, search, filters, and status views depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support dashboard analytics without introducing unnecessary architectural layers.

Design focus

- Dashboard analytics: Search, Filters, and Status Views is addressed through an observable user or administrator workflow.
- Resource filters: Search, Filters, and Status Views is addressed through a defined application or database boundary.
- Status badges: Search, Filters, and Status Views is addressed through an observable user or administrator workflow.

Acceptance evidence

- Role-specific navigation: Search, Filters, and Status Views is addressed through a defined application or database boundary.
- Resident self-service: Search, Filters, and Status Views is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Search, Filters, and Status Views is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 7

7.5 RESIDENT SELF-SERVICE

The finished feature set supports seven application roles: Admin, Committee, Secretary, Resident, Security, Housekeeping, and Maintenance Staff. Each role sees only relevant pages and actions, while record ownership further limits resident and staff data. This section focuses on resident self-service and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of resident self-service. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, resident self-service depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support resource filters without introducing unnecessary architectural layers.

Design focus

- Dashboard analytics: Resident Self-Service is addressed through an observable user or administrator workflow.
- Resource filters: Resident Self-Service is addressed through a defined application or database boundary.
- Status badges: Resident Self-Service is addressed through an observable user or administrator workflow.

Acceptance evidence

- Role-specific navigation: Resident Self-Service is addressed through a defined application or database boundary.
- Resident self-service: Resident Self-Service is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Resident Self-Service is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 7

7.6 ADMINISTRATIVE ANALYTICS

The finished feature set supports seven application roles: Admin, Committee, Secretary, Resident, Security, Housekeeping, and Maintenance Staff. Each role sees only relevant pages and actions, while record ownership further limits resident and staff data. This section focuses on administrative analytics and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of dashboard analytics. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, administrative analytics depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support status badges without introducing unnecessary architectural layers.

Design focus

- Dashboard analytics: Administrative Analytics is addressed through an observable user or administrator workflow.
- Resource filters: Administrative Analytics is addressed through a defined application or database boundary.
- Status badges: Administrative Analytics is addressed through an observable user or administrator workflow.

Acceptance evidence

- Role-specific navigation: Administrative Analytics is addressed through a defined application or database boundary.
- Resident self-service: Administrative Analytics is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Administrative Analytics is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 7

7.7 SECURITY AND DATA PROTECTION

The finished feature set supports seven application roles: Admin, Committee, Secretary, Resident, Security, Housekeeping, and Maintenance Staff. Each role sees only relevant pages and actions, while record ownership further limits resident and staff data. This section focuses on security and data protection and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of resource filters. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, security and data protection depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support role-specific navigation without introducing unnecessary architectural layers.

Design focus

- Dashboard analytics: Security and Data Protection is addressed through an observable user or administrator workflow.
- Resource filters: Security and Data Protection is addressed through a defined application or database boundary.
- Status badges: Security and Data Protection is addressed through an observable user or administrator workflow.

Acceptance evidence

- Role-specific navigation: Security and Data Protection is addressed through a defined application or database boundary.
- Resident self-service: Security and Data Protection is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Security and Data Protection is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 7

7.8 ACCESSIBILITY AND RESPONSIVE DESIGN

The finished feature set supports seven application roles: Admin, Committee, Secretary, Resident, Security, Housekeeping, and Maintenance Staff. Each role sees only relevant pages and actions, while record ownership further limits resident and staff data. This section focuses on accessibility and responsive design and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of status badges. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, accessibility and responsive design depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support resident self-service without introducing unnecessary architectural layers.

Design focus

- Dashboard analytics: Accessibility and Responsive Design is addressed through an observable user or administrator workflow.
- Resource filters: Accessibility and Responsive Design is addressed through a defined application or database boundary.
- Status badges: Accessibility and Responsive Design is addressed through an observable user or administrator workflow.

Acceptance evidence

- Role-specific navigation: Accessibility and Responsive Design is addressed through a defined application or database boundary.
- Resident self-service: Accessibility and Responsive Design is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Accessibility and Responsive Design is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 8

8.1 TESTING STRATEGY

Verification combines static checks, a production build, API scenarios, database constraints, and role-based negative tests. The repository includes a Postman collection for health, login, list, create, update, and delete behavior. Claims in this chapter distinguish implemented checks from recommended extensions. This section focuses on testing strategy and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of positive api paths. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, testing strategy depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support forbidden role actions without introducing unnecessary architectural layers.

ID	Scenario	Expected Result
T01	Health endpoint with reachable MySQL	200; database = connected
T02	Admin login with valid credentials	200; signed JWT and user payload
T03	Login with incorrect password	401; generic credential message
T04	Resident requests another resident's complaint	Record excluded by ownership scope
T05	Security role requests payments	403 Forbidden
T06	Create duplicate transaction ID	409 Conflict
T07	Delete referenced flat	409; referential-integrity message
T08	Create booking with end before start	Database check rejects write

Page outcome: Testing Strategy is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 8

8.2 POSTMAN API TESTING

Verification combines static checks, a production build, API scenarios, database constraints, and role-based negative tests. The repository includes a Postman collection for health, login, list, create, update, and delete behavior. Claims in this chapter distinguish implemented checks from recommended extensions. This section focuses on postman api testing and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of invalid credential paths. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, postman api testing depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support constraint violations without introducing unnecessary architectural layers.

ID	Scenario	Expected Result
T01	Health endpoint with reachable MySQL	200; database = connected
T02	Admin login with valid credentials	200; signed JWT and user payload
T03	Login with incorrect password	401; generic credential message
T04	Resident requests another resident's complaint	Record excluded by ownership scope
T05	Security role requests payments	403 Forbidden
T06	Create duplicate transaction ID	409 Conflict
T07	Delete referenced flat	409; referential-integrity message
T08	Create booking with end before start	Database check rejects write

Page outcome: Postman API Testing is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 8

8.3 AUTHENTICATION AND AUTHORIZATION TESTS

Verification combines static checks, a production build, API scenarios, database constraints, and role-based negative tests. The repository includes a Postman collection for health, login, list, create, update, and delete behavior. Claims in this chapter distinguish implemented checks from recommended extensions. This section focuses on authentication and authorization tests and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of forbidden role actions. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, authentication and authorization tests depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support regression build and lint checks without introducing unnecessary architectural layers.

ID	Scenario	Expected Result
T01	Health endpoint with reachable MySQL	200; database = connected
T02	Admin login with valid credentials	200; signed JWT and user payload
T03	Login with incorrect password	401; generic credential message
T04	Resident requests another resident's complaint	Record excluded by ownership scope
T05	Security role requests payments	403 Forbidden
T06	Create duplicate transaction ID	409 Conflict
T07	Delete referenced flat	409; referential-integrity message
T08	Create booking with end before start	Database check rejects write

Page outcome: Authentication and Authorization Tests is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

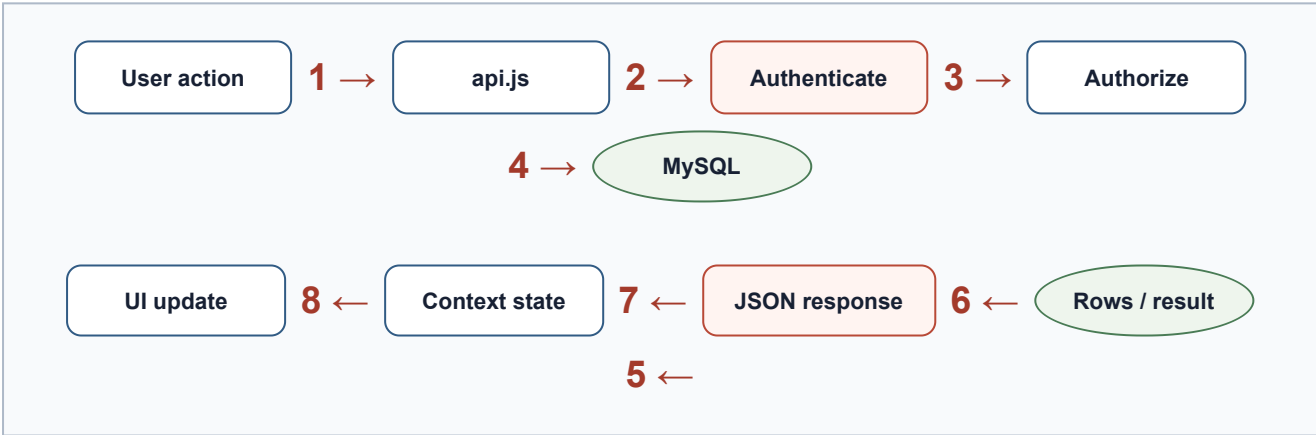
CHAPTER 8

8.4 INTEGRATION TESTING

Verification combines static checks, a production build, API scenarios, database constraints, and role-based negative tests. The repository includes a Postman collection for health, login, list, create, update, and delete behavior. Claims in this chapter distinguish implemented checks from recommended extensions. This section focuses on integration testing and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of constraint violations. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, integration testing depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support positive api paths without introducing unnecessary architectural layers.



Page outcome: Integration Testing is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 8

8.5 DATABASE CONSTRAINT TESTING

Verification combines static checks, a production build, API scenarios, database constraints, and role-based negative tests. The repository includes a Postman collection for health, login, list, create, update, and delete behavior. Claims in this chapter distinguish implemented checks from recommended extensions. This section focuses on database constraint testing and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of regression build and lint checks. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, database constraint testing depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support invalid credential paths without introducing unnecessary architectural layers.

ID	Scenario	Expected Result
T01	Health endpoint with reachable MySQL	200; database = connected
T02	Admin login with valid credentials	200; signed JWT and user payload
T03	Login with incorrect password	401; generic credential message
T04	Resident requests another resident's complaint	Record excluded by ownership scope
T05	Security role requests payments	403 Forbidden
T06	Create duplicate transaction ID	409 Conflict
T07	Delete referenced flat	409; referential-integrity message
T08	Create booking with end before start	Database check rejects write

Page outcome: Database Constraint Testing is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 8

8.6 FRONTEND AND RESPONSIVE TESTING

Verification combines static checks, a production build, API scenarios, database constraints, and role-based negative tests. The repository includes a Postman collection for health, login, list, create, update, and delete behavior. Claims in this chapter distinguish implemented checks from recommended extensions. This section focuses on frontend and responsive testing and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of positive api paths. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, frontend and responsive testing depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support forbidden role actions without introducing unnecessary architectural layers.

ID	Scenario	Expected Result
T01	Health endpoint with reachable MySQL	200; database = connected
T02	Admin login with valid credentials	200; signed JWT and user payload
T03	Login with incorrect password	401; generic credential message
T04	Resident requests another resident's complaint	Record excluded by ownership scope
T05	Security role requests payments	403 Forbidden
T06	Create duplicate transaction ID	409 Conflict
T07	Delete referenced flat	409; referential-integrity message
T08	Create booking with end before start	Database check rejects write

Page outcome: Frontend and Responsive Testing is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 8

8.7 TEST TOOLS AND ENVIRONMENTS

Verification combines static checks, a production build, API scenarios, database constraints, and role-based negative tests. The repository includes a Postman collection for health, login, list, create, update, and delete behavior. Claims in this chapter distinguish implemented checks from recommended extensions. This section focuses on test tools and environments and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of invalid credential paths. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, test tools and environments depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support constraint violations without introducing unnecessary architectural layers.

ID	Scenario	Expected Result
T01	Health endpoint with reachable MySQL	200; database = connected
T02	Admin login with valid credentials	200; signed JWT and user payload
T03	Login with incorrect password	401; generic credential message
T04	Resident requests another resident's complaint	Record excluded by ownership scope
T05	Security role requests payments	403 Forbidden
T06	Create duplicate transaction ID	409 Conflict
T07	Delete referenced flat	409; referential-integrity message
T08	Create booking with end before start	Database check rejects write

Page outcome: Test Tools and Environments is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 8

8.8 TEST CASES AND EXPECTED RESULTS

Verification combines static checks, a production build, API scenarios, database constraints, and role-based negative tests. The repository includes a Postman collection for health, login, list, create, update, and delete behavior. Claims in this chapter distinguish implemented checks from recommended extensions. This section focuses on test cases and expected results and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of forbidden role actions. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, test cases and expected results depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support regression build and lint checks without introducing unnecessary architectural layers.

ID	Scenario	Expected Result
T01	Health endpoint with reachable MySQL	200; database = connected
T02	Admin login with valid credentials	200; signed JWT and user payload
T03	Login with incorrect password	401; generic credential message
T04	Resident requests another resident's complaint	Record excluded by ownership scope
T05	Security role requests payments	403 Forbidden
T06	Create duplicate transaction ID	409 Conflict
T07	Delete referenced flat	409; referential-integrity message
T08	Create booking with end before start	Database check rejects write

Page outcome: Test Cases and Expected Results is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 8

8.9 QUALITY SUMMARY AND RESIDUAL RISKS

Verification combines static checks, a production build, API scenarios, database constraints, and role-based negative tests. The repository includes a Postman collection for health, login, list, create, update, and delete behavior. Claims in this chapter distinguish implemented checks from recommended extensions. This section focuses on quality summary and residual risks and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of constraint violations. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, quality summary and residual risks depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support positive api paths without introducing unnecessary architectural layers.

ID	Scenario	Expected Result
T01	Health endpoint with reachable MySQL	200; database = connected
T02	Admin login with valid credentials	200; signed JWT and user payload
T03	Login with incorrect password	401; generic credential message
T04	Resident requests another resident's complaint	Record excluded by ownership scope
T05	Security role requests payments	403 Forbidden
T06	Create duplicate transaction ID	409 Conflict
T07	Delete referenced flat	409; referential-integrity message
T08	Create booking with end before start	Database check rejects write

Page outcome: Quality Summary and Residual Risks is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 9

9.1 DEPLOYMENT OVERVIEW

Deployment separates the Vite frontend, Express API, and MySQL database. Configuration is injected through environment variables. The documented approach favors repeatable builds, additive database changes, health checks, backups, and rollback without destructive data loss. This section focuses on deployment overview and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of environment isolation. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, deployment overview depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support frontend static build without introducing unnecessary architectural layers.

Design focus

- Environment isolation: Deployment Overview is addressed through an observable user or administrator workflow.
- Schema and seed sequence: Deployment Overview is addressed through a defined application or database boundary.
- Frontend static build: Deployment Overview is addressed through an observable user or administrator workflow.

Acceptance evidence

- API process management: Deployment Overview is addressed through a defined application or database boundary.
- Backup and rollback: Deployment Overview is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Deployment Overview is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 9

9.2 LOCAL DATABASE PREPARATION

Deployment separates the Vite frontend, Express API, and MySQL database. Configuration is injected through environment variables. The documented approach favors repeatable builds, additive database changes, health checks, backups, and rollback without destructive data loss. This section focuses on local database preparation and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of schema and seed sequence. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, local database preparation depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support api process management without introducing unnecessary architectural layers.

Design focus

- Environment isolation: Local Database Preparation is addressed through an observable user or administrator workflow.
- Schema and seed sequence: Local Database Preparation is addressed through a defined application or database boundary.
- Frontend static build: Local Database Preparation is addressed through an observable user or administrator workflow.

Acceptance evidence

- API process management: Local Database Preparation is addressed through a defined application or database boundary.
- Backup and rollback: Local Database Preparation is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Local Database Preparation is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 9

9.3 ENVIRONMENT CONFIGURATION

Deployment separates the Vite frontend, Express API, and MySQL database. Configuration is injected through environment variables. The documented approach favors repeatable builds, additive database changes, health checks, backups, and rollback without destructive data loss. This section focuses on environment configuration and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of frontend static build. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, environment configuration depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support backup and rollback without introducing unnecessary architectural layers.

Variable	Purpose	Required
DB_HOST	MySQL host	Yes
DB_PORT	MySQL port; usually 3306	No
DB_USER	Least-privilege database account	Yes
DB_PASSWORD	Database password	Yes
DB_NAME	society_management	Yes
JWT_SECRET	Token signature secret	Yes
FRONTEND_URL	Allowed browser origin	Production
PORT	API listen port	No

Page outcome: Environment Configuration is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

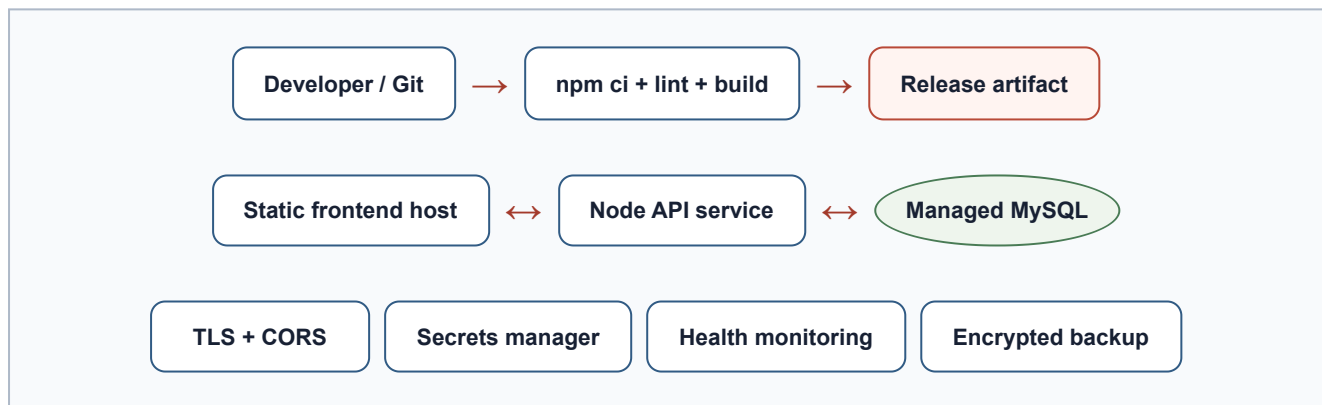
CHAPTER 9

9.4 FRONTEND PRODUCTION BUILD

Deployment separates the Vite frontend, Express API, and MySQL database. Configuration is injected through environment variables. The documented approach favors repeatable builds, additive database changes, health checks, backups, and rollback without destructive data loss. This section focuses on frontend production build and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of api process management. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, frontend production build depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support environment isolation without introducing unnecessary architectural layers.



Page outcome: Frontend Production Build is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 9

9.5 BACKEND SERVICE DEPLOYMENT

Deployment separates the Vite frontend, Express API, and MySQL database. Configuration is injected through environment variables. The documented approach favors repeatable builds, additive database changes, health checks, backups, and rollback without destructive data loss. This section focuses on backend service deployment and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of backup and rollback. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, backend service deployment depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support schema and seed sequence without introducing unnecessary architectural layers.

Design focus

- Environment isolation: Backend Service Deployment is addressed through an observable user or administrator workflow.
- Schema and seed sequence: Backend Service Deployment is addressed through a defined application or database boundary.
- Frontend static build: Backend Service Deployment is addressed through an observable user or administrator workflow.

Acceptance evidence

- API process management: Backend Service Deployment is addressed through a defined application or database boundary.
- Backup and rollback: Backend Service Deployment is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Backend Service Deployment is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 9

9.6 DATABASE MIGRATION AND BACKUP

Deployment separates the Vite frontend, Express API, and MySQL database. Configuration is injected through environment variables. The documented approach favors repeatable builds, additive database changes, health checks, backups, and rollback without destructive data loss. This section focuses on database migration and backup and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of environment isolation. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, database migration and backup depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support frontend static build without introducing unnecessary architectural layers.

Design focus

- Environment isolation: Database Migration and Backup is addressed through an observable user or administrator workflow.
- Schema and seed sequence: Database Migration and Backup is addressed through a defined application or database boundary.
- Frontend static build: Database Migration and Backup is addressed through an observable user or administrator workflow.

Acceptance evidence

- API process management: Database Migration and Backup is addressed through a defined application or database boundary.
- Backup and rollback: Database Migration and Backup is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Database Migration and Backup is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 9

9.7 CI/CD PIPELINE DESIGN

Deployment separates the Vite frontend, Express API, and MySQL database. Configuration is injected through environment variables. The documented approach favors repeatable builds, additive database changes, health checks, backups, and rollback without destructive data loss. This section focuses on ci/cd pipeline design and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of schema and seed sequence. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, ci/cd pipeline design depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support api process management without introducing unnecessary architectural layers.

Design focus • Environment isolation: CI/CD Pipeline Design is addressed through an observable user or administrator workflow. • Schema and seed sequence: CI/CD Pipeline Design is addressed through a defined application or database boundary. • Frontend static build: CI/CD Pipeline Design is addressed through an observable user or administrator workflow.	Acceptance evidence • API process management: CI/CD Pipeline Design is addressed through a defined application or database boundary. • Backup and rollback: CI/CD Pipeline Design is addressed through an observable user or administrator workflow. • Repository anchor: README.md and project configuration.
---	--

Page outcome: CI/CD Pipeline Design is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 9

9.8 SECURITY HARDENING AND MONITORING

Deployment separates the Vite frontend, Express API, and MySQL database. Configuration is injected through environment variables. The documented approach favors repeatable builds, additive database changes, health checks, backups, and rollback without destructive data loss. This section focuses on security hardening and monitoring and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of frontend static build. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, security hardening and monitoring depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support backup and rollback without introducing unnecessary architectural layers.

Design focus

- Environment isolation: Security Hardening and Monitoring is addressed through an observable user or administrator workflow.
- Schema and seed sequence: Security Hardening and Monitoring is addressed through a defined application or database boundary.
- Frontend static build: Security Hardening and Monitoring is addressed through an observable user or administrator workflow.

Acceptance evidence

- API process management: Security Hardening and Monitoring is addressed through a defined application or database boundary.
- Backup and rollback: Security Hardening and Monitoring is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Security Hardening and Monitoring is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 9

9.9 ROLLBACK AND RECOVERY PROCEDURE

Deployment separates the Vite frontend, Express API, and MySQL database. Configuration is injected through environment variables. The documented approach favors repeatable builds, additive database changes, health checks, backups, and rollback without destructive data loss. This section focuses on rollback and recovery procedure and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of api process management. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, rollback and recovery procedure depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support environment isolation without introducing unnecessary architectural layers.

Design focus • Environment isolation: Rollback and Recovery Procedure is addressed through an observable user or administrator workflow. • Schema and seed sequence: Rollback and Recovery Procedure is addressed through a defined application or database boundary. • Frontend static build: Rollback and Recovery Procedure is addressed through an observable user or administrator workflow.	Acceptance evidence • API process management: Rollback and Recovery Procedure is addressed through a defined application or database boundary. • Backup and rollback: Rollback and Recovery Procedure is addressed through an observable user or administrator workflow. • Repository anchor: README.md and project configuration.
---	--

Page outcome: Rollback and Recovery Procedure is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 10

10.1 ISSUES FACED DURING DEVELOPMENT

The central engineering challenges were maintaining mirrored navigation and server permissions, scoping data by resident ownership, preserving relational integrity, and keeping generic CRUD behavior understandable. The remaining limitations are recorded without presenting planned services as completed features. This section focuses on issues faced during development and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of permission drift. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, issues faced during development depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support foreign-key conflicts without introducing unnecessary architectural layers.

Design focus

- Permission drift: Issues Faced During Development is addressed through an observable user or administrator workflow.
- Stale client state: Issues Faced During Development is addressed through a defined application or database boundary.
- Foreign-key conflicts: Issues Faced During Development is addressed through an observable user or administrator workflow.

Acceptance evidence

- Environment mismatch: Issues Faced During Development is addressed through a defined application or database boundary.
- Limited automation: Issues Faced During Development is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Issues Faced During Development is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 10

10.2 RELATIONAL DATA AND REFERENTIAL INTEGRITY

The central engineering challenges were maintaining mirrored navigation and server permissions, scoping data by resident ownership, preserving relational integrity, and keeping generic CRUD behavior understandable. The remaining limitations are recorded without presenting planned services as completed features. This section focuses on relational data and referential integrity and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of stale client state. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, relational data and referential integrity depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support environment mismatch without introducing unnecessary architectural layers.

Design focus

- Permission drift: Relational Data and Referential Integrity is addressed through an observable user or administrator workflow.
- Stale client state: Relational Data and Referential Integrity is addressed through a defined application or database boundary.
- Foreign-key conflicts: Relational Data and Referential Integrity is addressed through an observable user or administrator workflow.

Acceptance evidence

- Environment mismatch: Relational Data and Referential Integrity is addressed through a defined application or database boundary.
- Limited automation: Relational Data and Referential Integrity is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Relational Data and Referential Integrity is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 10

10.3 ROLE PERMISSION COMPLEXITY

The central engineering challenges were maintaining mirrored navigation and server permissions, scoping data by resident ownership, preserving relational integrity, and keeping generic CRUD behavior understandable. The remaining limitations are recorded without presenting planned services as completed features. This section focuses on role permission complexity and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of foreign-key conflicts. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, role permission complexity depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support limited automation without introducing unnecessary architectural layers.

Design focus

- **Permission drift: Role Permission**
Complexity is addressed through an observable user or administrator workflow.
- **Stale client state: Role Permission**
Complexity is addressed through a defined application or database boundary.
- **Foreign-key conflicts: Role Permission**
Complexity is addressed through an observable user or administrator workflow.

Acceptance evidence

- **Environment mismatch: Role Permission**
Complexity is addressed through a defined application or database boundary.
- **Limited automation: Role Permission**
Complexity is addressed through an observable user or administrator workflow.
- **Repository anchor: README.md and project configuration.**

Page outcome: Role Permission Complexity is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 10

10.4 FRONTEND STATE SYNCHRONIZATION

The central engineering challenges were maintaining mirrored navigation and server permissions, scoping data by resident ownership, preserving relational integrity, and keeping generic CRUD behavior understandable. The remaining limitations are recorded without presenting planned services as completed features. This section focuses on frontend state synchronization and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of environment mismatch. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, frontend state synchronization depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support permission drift without introducing unnecessary architectural layers.

Design focus

- **Permission drift:** Frontend State Synchronization is addressed through an observable user or administrator workflow.
- **Stale client state:** Frontend State Synchronization is addressed through a defined application or database boundary.
- **Foreign-key conflicts:** Frontend State Synchronization is addressed through an observable user or administrator workflow.

Acceptance evidence

- **Environment mismatch:** Frontend State Synchronization is addressed through a defined application or database boundary.
- **Limited automation:** Frontend State Synchronization is addressed through an observable user or administrator workflow.
- **Repository anchor:** README.md and project configuration.

Page outcome: Frontend State Synchronization is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 10

10.5 AUTHENTICATION AND SESSION CHALLENGES

The central engineering challenges were maintaining mirrored navigation and server permissions, scoping data by resident ownership, preserving relational integrity, and keeping generic CRUD behavior understandable. The remaining limitations are recorded without presenting planned services as completed features. This section focuses on authentication and session challenges and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of limited automation. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, authentication and session challenges depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support stale client state without introducing unnecessary architectural layers.

Design focus

- Permission drift: Authentication and Session Challenges is addressed through an observable user or administrator workflow.
- Stale client state: Authentication and Session Challenges is addressed through a defined application or database boundary.
- Foreign-key conflicts: Authentication and Session Challenges is addressed through an observable user or administrator workflow.

Acceptance evidence

- Environment mismatch: Authentication and Session Challenges is addressed through a defined application or database boundary.
- Limited automation: Authentication and Session Challenges is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Authentication and Session Challenges is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 10

10.6 SOLUTIONS APPLIED

The central engineering challenges were maintaining mirrored navigation and server permissions, scoping data by resident ownership, preserving relational integrity, and keeping generic CRUD behavior understandable. The remaining limitations are recorded without presenting planned services as completed features. This section focuses on solutions applied and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of permission drift. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, solutions applied depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support foreign-key conflicts without introducing unnecessary architectural layers.

Design focus

- Permission drift: Solutions Applied is addressed through an observable user or administrator workflow.
- Stale client state: Solutions Applied is addressed through a defined application or database boundary.
- Foreign-key conflicts: Solutions Applied is addressed through an observable user or administrator workflow.

Acceptance evidence

- Environment mismatch: Solutions Applied is addressed through a defined application or database boundary.
- Limited automation: Solutions Applied is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Solutions Applied is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 10

10.7 CURRENT LIMITATIONS

The central engineering challenges were maintaining mirrored navigation and server permissions, scoping data by resident ownership, preserving relational integrity, and keeping generic CRUD behavior understandable. The remaining limitations are recorded without presenting planned services as completed features. This section focuses on current limitations and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of stale client state. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, current limitations depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support environment mismatch without introducing unnecessary architectural layers.

Design focus

- Permission drift: Current Limitations is addressed through an observable user or administrator workflow.
- Stale client state: Current Limitations is addressed through a defined application or database boundary.
- Foreign-key conflicts: Current Limitations is addressed through an observable user or administrator workflow.

Acceptance evidence

- Environment mismatch: Current Limitations is addressed through a defined application or database boundary.
- Limited automation: Current Limitations is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Current Limitations is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 10

10.8 KNOWN RISKS AND MITIGATIONS

The central engineering challenges were maintaining mirrored navigation and server permissions, scoping data by resident ownership, preserving relational integrity, and keeping generic CRUD behavior understandable. The remaining limitations are recorded without presenting planned services as completed features. This section focuses on known risks and mitigations and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of foreign-key conflicts. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, known risks and mitigations depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support limited automation without introducing unnecessary architectural layers.

Design focus

- Permission drift: Known Risks and Mitigations is addressed through an observable user or administrator workflow.
- Stale client state: Known Risks and Mitigations is addressed through a defined application or database boundary.
- Foreign-key conflicts: Known Risks and Mitigations is addressed through an observable user or administrator workflow.

Acceptance evidence

- Environment mismatch: Known Risks and Mitigations is addressed through a defined application or database boundary.
- Limited automation: Known Risks and Mitigations is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Known Risks and Mitigations is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 10

10.9 CHAPTER SUMMARY

The central engineering challenges were maintaining mirrored navigation and server permissions, scoping data by resident ownership, preserving relational integrity, and keeping generic CRUD behavior understandable. The remaining limitations are recorded without presenting planned services as completed features. This section focuses on chapter summary and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of environment mismatch. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, chapter summary depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support permission drift without introducing unnecessary architectural layers.

Design focus

- Permission drift: Chapter Summary is addressed through an observable user or administrator workflow.
- Stale client state: Chapter Summary is addressed through a defined application or database boundary.
- Foreign-key conflicts: Chapter Summary is addressed through an observable user or administrator workflow.

Acceptance evidence

- Environment mismatch: Chapter Summary is addressed through a defined application or database boundary.
- Limited automation: Chapter Summary is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Chapter Summary is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 11

11.1 PLANNED FEATURES

Future work is arranged as backward-compatible increments. New schema fields should begin nullable or defaulted, behavior changes should be feature-gated, and existing API contracts should remain available through a migration period. This section focuses on planned features and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of payment integration. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, planned features depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support pwa installation without introducing unnecessary architectural layers.

Design focus

- Payment integration: Planned Features is addressed through an observable user or administrator workflow.
- Email and SMS alerts: Planned Features is addressed through a defined application or database boundary.
- PWA installation: Planned Features is addressed through an observable user or administrator workflow.

Acceptance evidence

- Audit history: Planned Features is addressed through a defined application or database boundary.
- Advanced trend analytics: Planned Features is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Planned Features is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 11

11.2 DIGITAL PAYMENT GATEWAY INTEGRATION

Future work is arranged as backward-compatible increments. New schema fields should begin nullable or defaulted, behavior changes should be feature-gated, and existing API contracts should remain available through a migration period. This section focuses on digital payment gateway integration and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of email and sms alerts. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, digital payment gateway integration depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support audit history without introducing unnecessary architectural layers.

Design focus • Payment integration: Digital Payment Gateway Integration is addressed through an observable user or administrator workflow. • Email and SMS alerts: Digital Payment Gateway Integration is addressed through a defined application or database boundary. • PWA installation: Digital Payment Gateway Integration is addressed through an observable user or administrator workflow.	Acceptance evidence • Audit history: Digital Payment Gateway Integration is addressed through a defined application or database boundary. • Advanced trend analytics: Digital Payment Gateway Integration is addressed through an observable user or administrator workflow. • Repository anchor: README.md and project configuration.
--	--

Page outcome: Digital Payment Gateway Integration is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 11

11.3 NOTIFICATION AND COMMUNICATION SERVICES

Future work is arranged as backward-compatible increments. New schema fields should begin nullable or defaulted, behavior changes should be feature-gated, and existing API contracts should remain available through a migration period. This section focuses on notification and communication services and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of pwa installation. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, notification and communication services depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support advanced trend analytics without introducing unnecessary architectural layers.

Design focus

- Payment integration: Notification and Communication Services is addressed through an observable user or administrator workflow.
- Email and SMS alerts: Notification and Communication Services is addressed through a defined application or database boundary.
- PWA installation: Notification and Communication Services is addressed through an observable user or administrator workflow.

Acceptance evidence

- Audit history: Notification and Communication Services is addressed through a defined application or database boundary.
- Advanced trend analytics: Notification and Communication Services is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Notification and Communication Services is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 11

11.4 MOBILE AND PROGRESSIVE WEB APPLICATION

Future work is arranged as backward-compatible increments. New schema fields should begin nullable or defaulted, behavior changes should be feature-gated, and existing API contracts should remain available through a migration period. This section focuses on mobile and progressive web application and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of audit history. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, mobile and progressive web application depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support payment integration without introducing unnecessary architectural layers.

Design focus

- Payment integration: Mobile and Progressive Web Application is addressed through an observable user or administrator workflow.
- Email and SMS alerts: Mobile and Progressive Web Application is addressed through a defined application or database boundary.
- PWA installation: Mobile and Progressive Web Application is addressed through an observable user or administrator workflow.

Acceptance evidence

- Audit history: Mobile and Progressive Web Application is addressed through a defined application or database boundary.
- Advanced trend analytics: Mobile and Progressive Web Application is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Mobile and Progressive Web Application is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 11

11.5 ADVANCED ANALYTICS AND AUTOMATION

Future work is arranged as backward-compatible increments. New schema fields should begin nullable or defaulted, behavior changes should be feature-gated, and existing API contracts should remain available through a migration period. This section focuses on advanced analytics and automation and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of advanced trend analytics. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, advanced analytics and automation depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support email and sms alerts without introducing unnecessary architectural layers.

Design focus

- Payment integration: Advanced Analytics and Automation is addressed through an observable user or administrator workflow.
- Email and SMS alerts: Advanced Analytics and Automation is addressed through a defined application or database boundary.
- PWA installation: Advanced Analytics and Automation is addressed through an observable user or administrator workflow.

Acceptance evidence

- Audit history: Advanced Analytics and Automation is addressed through a defined application or database boundary.
- Advanced trend analytics: Advanced Analytics and Automation is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Advanced Analytics and Automation is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 11

11.6 INTEGRATION AND OPTIMIZATION ROADMAP

Future work is arranged as backward-compatible increments. New schema fields should begin nullable or defaulted, behavior changes should be feature-gated, and existing API contracts should remain available through a migration period. This section focuses on integration and optimization roadmap and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of payment integration. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, integration and optimization roadmap depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support pwa installation without introducing unnecessary architectural layers.

Phase 1	Audit logs, automated tests, accessibility improvements
Phase 2	Email/SMS notifications and payment gateway behind feature flags
Phase 3	PWA support, offline read-only views, export and reporting
Phase 4	Predictive maintenance, anomaly detection, multi-society tenancy

Page outcome: Integration and Optimization Roadmap is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 12

12.1 PROJECT SUMMARY

The project demonstrates a complete database-backed information system rather than a static interface. Its strongest outcome is a coherent path from schema constraints and API authorization to role-aware pages and analytics. This section focuses on project summary and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of full-stack integration. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, project summary depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support secure authentication without introducing unnecessary architectural layers.

Design focus

- Full-stack integration: Project Summary is addressed through an observable user or administrator workflow.
- Database design: Project Summary is addressed through a defined application or database boundary.
- Secure authentication: Project Summary is addressed through an observable user or administrator workflow.

Acceptance evidence

- Reusable UI engineering: Project Summary is addressed through a defined application or database boundary.
- Testing discipline: Project Summary is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Project Summary is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 12

12.2 OBJECTIVES ACHIEVED

The project demonstrates a complete database-backed information system rather than a static interface. Its strongest outcome is a coherent path from schema constraints and API authorization to role-aware pages and analytics. This section focuses on objectives achieved and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of database design. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, objectives achieved depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support reusable ui engineering without introducing unnecessary architectural layers.

Design focus

- Full-stack integration: Objectives Achieved is addressed through an observable user or administrator workflow.
- Database design: Objectives Achieved is addressed through a defined application or database boundary.
- Secure authentication: Objectives Achieved is addressed through an observable user or administrator workflow.

Acceptance evidence

- Reusable UI engineering: Objectives Achieved is addressed through a defined application or database boundary.
- Testing discipline: Objectives Achieved is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Objectives Achieved is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 12

12.3 TECHNICAL ACHIEVEMENTS

The project demonstrates a complete database-backed information system rather than a static interface. Its strongest outcome is a coherent path from schema constraints and API authorization to role-aware pages and analytics. This section focuses on technical achievements and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of secure authentication. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, technical achievements depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support testing discipline without introducing unnecessary architectural layers.

Design focus • Full-stack integration: Technical Achievements is addressed through an observable user or administrator workflow. • Database design: Technical Achievements is addressed through a defined application or database boundary. • Secure authentication: Technical Achievements is addressed through an observable user or administrator workflow.	Acceptance evidence • Reusable UI engineering: Technical Achievements is addressed through a defined application or database boundary. • Testing discipline: Technical Achievements is addressed through an observable user or administrator workflow. • Repository anchor: README.md and project configuration.
--	--

Page outcome: Technical Achievements is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 12

12.4 FUNCTIONAL OUTCOMES

The project demonstrates a complete database-backed information system rather than a static interface. Its strongest outcome is a coherent path from schema constraints and API authorization to role-aware pages and analytics. This section focuses on functional outcomes and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of reusable ui engineering. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, functional outcomes depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support full-stack integration without introducing unnecessary architectural layers.

Design focus

- Full-stack integration: Functional Outcomes is addressed through an observable user or administrator workflow.
- Database design: Functional Outcomes is addressed through a defined application or database boundary.
- Secure authentication: Functional Outcomes is addressed through an observable user or administrator workflow.

Acceptance evidence

- Reusable UI engineering: Functional Outcomes is addressed through a defined application or database boundary.
- Testing discipline: Functional Outcomes is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Functional Outcomes is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 12

12.5 SECURITY AND RELIABILITY OUTCOMES

The project demonstrates a complete database-backed information system rather than a static interface. Its strongest outcome is a coherent path from schema constraints and API authorization to role-aware pages and analytics. This section focuses on security and reliability outcomes and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of testing discipline. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, security and reliability outcomes depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support database design without introducing unnecessary architectural layers.

Design focus

- Full-stack integration: Security and Reliability Outcomes is addressed through an observable user or administrator workflow.
- Database design: Security and Reliability Outcomes is addressed through a defined application or database boundary.
- Secure authentication: Security and Reliability Outcomes is addressed through an observable user or administrator workflow.

Acceptance evidence

- Reusable UI engineering: Security and Reliability Outcomes is addressed through a defined application or database boundary.
- Testing discipline: Security and Reliability Outcomes is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Security and Reliability Outcomes is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 12

12.6 USER EXPERIENCE OUTCOMES

The project demonstrates a complete database-backed information system rather than a static interface. Its strongest outcome is a coherent path from schema constraints and API authorization to role-aware pages and analytics. This section focuses on user experience outcomes and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of full-stack integration. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, user experience outcomes depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support secure authentication without introducing unnecessary architectural layers.

Design focus

- Full-stack integration: User Experience Outcomes is addressed through an observable user or administrator workflow.
- Database design: User Experience Outcomes is addressed through a defined application or database boundary.
- Secure authentication: User Experience Outcomes is addressed through an observable user or administrator workflow.

Acceptance evidence

- Reusable UI engineering: User Experience Outcomes is addressed through a defined application or database boundary.
- Testing discipline: User Experience Outcomes is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: User Experience Outcomes is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 12

12.7 SKILLS LEARNED DURING DEVELOPMENT

The project demonstrates a complete database-backed information system rather than a static interface. Its strongest outcome is a coherent path from schema constraints and API authorization to role-aware pages and analytics. This section focuses on skills learned during development and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of database design. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, skills learned during development depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support reusable ui engineering without introducing unnecessary architectural layers.

Design focus

- Full-stack integration: Skills Learned During Development is addressed through an observable user or administrator workflow.
- Database design: Skills Learned During Development is addressed through a defined application or database boundary.
- Secure authentication: Skills Learned During Development is addressed through an observable user or administrator workflow.

Acceptance evidence

- Reusable UI engineering: Skills Learned During Development is addressed through a defined application or database boundary.
- Testing discipline: Skills Learned During Development is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Skills Learned During Development is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 12

12.8 SOCIAL AND OPERATIONAL IMPACT

The project demonstrates a complete database-backed information system rather than a static interface. Its strongest outcome is a coherent path from schema constraints and API authorization to role-aware pages and analytics. This section focuses on social and operational impact and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of secure authentication. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, social and operational impact depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support testing discipline without introducing unnecessary architectural layers.

Design focus

- Full-stack integration: Social and Operational Impact is addressed through an observable user or administrator workflow.
- Database design: Social and Operational Impact is addressed through a defined application or database boundary.
- Secure authentication: Social and Operational Impact is addressed through an observable user or administrator workflow.

Acceptance evidence

- Reusable UI engineering: Social and Operational Impact is addressed through a defined application or database boundary.
- Testing discipline: Social and Operational Impact is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Social and Operational Impact is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 12

12.9 FINAL CONCLUSION

The project demonstrates a complete database-backed information system rather than a static interface. Its strongest outcome is a coherent path from schema constraints and API authorization to role-aware pages and analytics. This section focuses on final conclusion and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of reusable ui engineering. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, final conclusion depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support full-stack integration without introducing unnecessary architectural layers.

Design focus

- Full-stack integration: Final Conclusion is addressed through an observable user or administrator workflow.
- Database design: Final Conclusion is addressed through a defined application or database boundary.
- Secure authentication: Final Conclusion is addressed through an observable user or administrator workflow.

Acceptance evidence

- Reusable UI engineering: Final Conclusion is addressed through a defined application or database boundary.
- Testing discipline: Final Conclusion is addressed through an observable user or administrator workflow.
- Repository anchor: README.md and project configuration.

Page outcome: Final Conclusion is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 13

13.1 BOOKS AND STANDARDS

References prioritize official documentation and established software engineering sources. Version-specific product documentation is listed so that technical statements can be checked against the implemented dependency set. This section focuses on books and standards and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of evidence-based implementation. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, books and standards depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support operational clarity without introducing unnecessary architectural layers.

Published Sources and Standards

1. Pressman, R. S. and Maxim, B. R. Software Engineering: A Practitioner's Approach. McGraw-Hill.
2. Sommerville, I. Software Engineering. Pearson.
3. OWASP Foundation. OWASP Application Security Verification Standard and Web Security Testing Guide.
4. Fielding, R. Architectural Styles and the Design of Network-based Software Architectures.
5. Oracle. MySQL 8.0 Reference Manual: InnoDB, constraints, transactions, and indexing.
6. ECMA International. ECMAScript Language Specification.

All URLs were last reviewed for this report on 2 October 2026. Dependency versions are recorded in the repository package manifest and lock file.

Page outcome: Books and Standards is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 13

13.2 DOCUMENTATION AND TECHNICAL RESOURCES

References prioritize official documentation and established software engineering sources. Version-specific product documentation is listed so that technical statements can be checked against the implemented dependency set. This section focuses on documentation and technical resources and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of maintainable design. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, documentation and technical resources depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support secure data handling without introducing unnecessary architectural layers.

Framework and Tool Documentation

1. React Team. React Documentation. <https://react.dev/>
2. Vite Team. Vite Guide. <https://vite.dev/>
3. Express.js. Express 5 Documentation. <https://expressjs.com/>
4. MySQL2. Node.js MySQL driver documentation. <https://sidorares.github.io/node-mysql2/>
5. Auth0. JSON Web Token introduction. <https://jwt.io/introduction>
6. Postman. Collections and test scripts documentation. <https://learning.postman.com/>
7. Recharts. Composable charting library documentation. <https://recharts.org/>
8. MDN Web Docs. HTTP, CORS, accessibility, and responsive design references. <https://developer.mozilla.org/>

All URLs were last reviewed for this report on 2 October 2026. Dependency versions are recorded in the repository package manifest and lock file.

Page outcome: Documentation and Technical Resources is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 14

14.1 APPLICATION SCREENSHOTS

The appendices provide practical evidence and operating guidance: screenshot locations, representative source excerpts, setup commands, user workflows, endpoint summaries, and signature placeholders. This section focuses on application screenshots and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of evidence-based implementation. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, application screenshots depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support operational clarity without introducing unnecessary architectural layers.

Application Screenshots

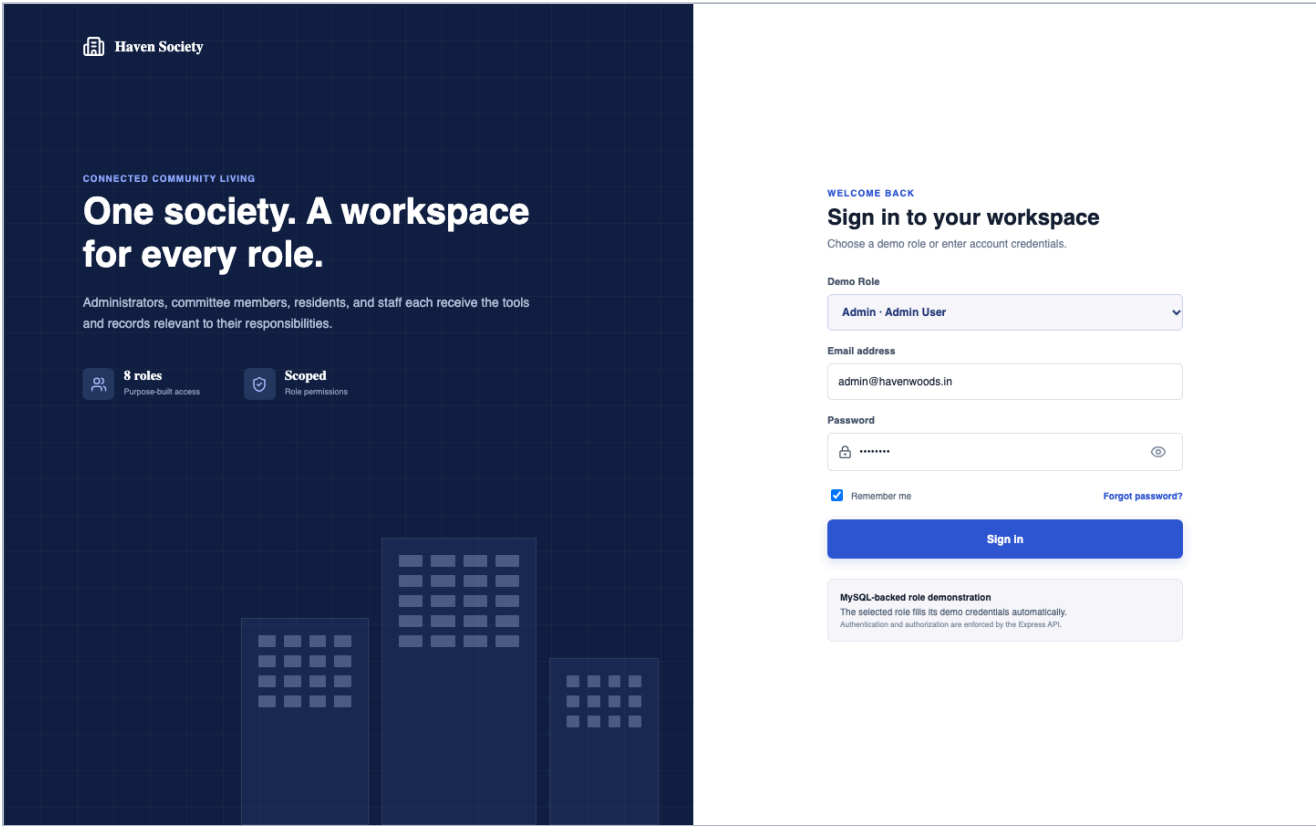


Figure A.1 — Responsive Haven Society login page and demo-role selector, captured from the running Vite application.

Figure A.2 Administrator dashboard with charts	Figure A.3 Resident complaint or booking view
---	--

CHAPTER 14

14.2 SAMPLE CODE SNIPPETS

The appendices provide practical evidence and operating guidance: screenshot locations, representative source excerpts, setup commands, user workflows, endpoint summaries, and signature placeholders. This section focuses on sample code snippets and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of maintainable design. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, sample code snippets depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support secure data handling without introducing unnecessary architectural layers.

Representative Source Code

Authenticated route boundary

```
app.use('/api/:resource', authenticate, (req, res, next) => {
  const config = resources[req.params.resource]
  if (!config) return res.status(404).json({ message: 'Unknown resource.' })
  if (!canAccess(req.user.role, req.params.resource))
    return res.status(403).json({ message: 'Access denied.' })
  req.resourceConfig = config
  next()
})
```

Parameterized list query

```
const scope = scopedWhere(req.user, req.params.resource)
const [rows] = await pool.execute(
  'SELECT * FROM ' + table + scope.sql + ' ORDER BY ' + id + ' DESC',
  scope.values,
)
```

The table and identifier originate only from server-owned resource configuration; user values remain bound parameters.

Page outcome: Sample Code Snippets is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 14

14.3 INSTALLATION AND SETUP INSTRUCTIONS

The appendices provide practical evidence and operating guidance: screenshot locations, representative source excerpts, setup commands, user workflows, endpoint summaries, and signature placeholders. This section focuses on installation and setup instructions and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of operational clarity. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, installation and setup instructions depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support future extensibility without introducing unnecessary architectural layers.

Installation and Setup

1. Install Node.js 20 or later and MySQL 8.
2. Run **npm install** from the repository root.
3. Create the database using **mysql -u root -p < backend/sql/schema.sql**.
4. Load demonstration data with **mysql -u root -p < backend/sql/seed.sql**.
5. Copy **.env.example** to **.env**; set **DB_PASSWORD** and **JWT_SECRET**.
6. Start the API using **npm run dev:server**.
7. Start Vite using **npm run dev**.
8. Open **http://localhost:5174** and verify **http://localhost:3001/api/health**.

```
npm install
npm run lint
npm run build
npm run dev:server
npm run dev
```

The frontend port is intentionally fixed at 5174. The default API port is 3001.

Page outcome: Installation and Setup Instructions is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 14

14.4 USER MANUAL

The appendices provide practical evidence and operating guidance: screenshot locations, representative source excerpts, setup commands, user workflows, endpoint summaries, and signature placeholders. This section focuses on user manual and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of secure data handling. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, user manual depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support evidence-based implementation without introducing unnecessary architectural layers.

User Manual

1. Sign in	Select a demo role or enter an authorized email and password.
2. Review dashboard	Use the role-specific summary to identify pending work.
3. Open a module	Choose a permitted resource from the sidebar.
4. Search and filter	Narrow table records by configured fields and statuses.
5. Create or edit	Complete the modal form; required fields are validated before submission.
6. Resolve work	Update complaint, visitor, booking, or billing status according to responsibility.
7. Sign out	Remove the local session token when leaving a shared device.

Authorization is enforced by the API even if a user attempts to bypass hidden navigation controls.

Page outcome: User Manual is documented as part of the implemented system and is traceable to a code, schema, configuration, or test artifact.

CHAPTER 14

14.5 API CATALOGUE AND TEST DATA

The appendices provide practical evidence and operating guidance: screenshot locations, representative source excerpts, setup commands, user workflows, endpoint summaries, and signature placeholders. This section focuses on api catalogue and test data and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of future extensibility. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, api catalogue and test data depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support maintainable design without introducing unnecessary architectural layers.

API Catalogue and Demonstration Data

Method	Endpoint	Purpose
GET	/api/health	API and database health
POST	/api/auth/login	Credential verification and JWT issue
GET	/api/:resource	Role-scoped record list
POST	/api/:resource	Create an allowed record
PUT	/api/:resource/:id	Update an allowed record
DELETE	/api/:resource/:id	Delete an allowed record

Supported resources

- flats
- residents
- visitors
- staff
- maintenance
- payments
- complaints
- notices
- amenities
- bookings

The included Postman collection automatically stores the admin JWT and exercises a notice create, update, and delete cycle.

CHAPTER 14

14.6 REVIEW FORMS AND SIGNATURES

The appendices provide practical evidence and operating guidance: screenshot locations, representative source excerpts, setup commands, user workflows, endpoint summaries, and signature placeholders. This section focuses on review forms and signatures and relates that subject directly to the implemented Haven codebase.

The design decision is evaluated through the principle of evidence-based implementation. Browser controls improve usability, but the API remains the trust boundary. Requests are accepted only after identity, role, resource, and—where applicable—record ownership have been evaluated. This avoids treating hidden buttons as a security mechanism.

From a database perspective, review forms and signatures depends on explicit keys, constrained status values, and predictable relationships. The use of MySQL transactions for multi-record operations prevents partial writes, while parameter binding protects user values from being interpreted as SQL. These choices support operational clarity without introducing unnecessary architectural layers.

Review Forms and Signatures

Attach signed review form or geo-tagged review photograph here.

Do not publish phone numbers, signatures, or location metadata outside the submitted academic copy.

Review date	
Reviewer / Guide	
Comments addressed	
Student signatures	